PLAYWRIGHT

CHEAT SHEETS

A Complete Reference for End-to-End Testing with Playwright

Locators · Assertions · Fixtures · Network & API · Visual & A11y · Debugging · CI & Sharding · Web Frameworks · Python · Java · .NET · AI Agents & MCP

67 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP
01 Introduction & Architecture
02 Installation
03 Project Structure
04 CLI Essentials
05 VS Code Extension

WRITING TESTS
06 Test Structure
07 Annotations & Tags
08 Test Steps & Test Info
09 Parameterized Tests

LOCATORS
10 getByRole
11 Other Built-in Locators
12 Filtering, Chaining & Raw Locators
13 Frames, Shadow DOM & Lists

ACTIONS
14 Clicks & Text Input
15 Forms & Files
16 Keyboard, Mouse & Drag
17 Dialogs, Downloads & New Pages

ASSERTIONS
18 Web-First Locator Assertions
19 Page & Generic Assertions
20 Soft, Polling & Custom Assertions

NAVIGATION & WAITING
21 Navigation & Load States
22 Auto-Waiting & Timeouts

FIXTURES & PAGE OBJECTS
23 Built-in Fixtures
24 Custom Fixtures
25 Page Object Model
26 Global Setup & Project Dependencies

AUTHENTICATION
27 Auth with storageState
28 Multi-Role & Advanced Auth

NETWORK
29 Routing, Mocking & HAR
30 Waiting on Network Traffic
31 API Testing

BROWSERS, CONTEXTS & EMULATION
32 Browsers & Channels
33 Devices & Emulation
34 Clock, Permissions & Storage

VISUAL & ACCESSIBILITY TESTING
35 Visual Comparisons & Snapshots
36 Accessibility & ARIA Snapshots

CONFIGURATION & RUNNING TESTS
37 playwright.config.ts
38 Parallelism & Sharding
39 Retries & Flaky Tests
40 Reporters
41 CLI Filtering & Flags

DEBUGGING & TOOLING
42 UI Mode
43 Trace Viewer
44 Inspector & Debug Mode
45 Codegen
46 Screenshots, Video & Screencast

AI AGENTS & MCP
47 Test Agents
48 Playwright MCP & playwright-cli

WEB APP FRAMEWORKS
49 webServer & baseURL
50 React (Vite) Apps
51 Next.js Apps
52 Vue, Angular & Svelte Apps
53 Component Testing — Setup
54 Component Testing — Stories & mount

LANGUAGE BINDINGS
55 Python — Setup, Sync & Async API
56 Python — pytest Plugin
57 Java — Setup, JUnit & TestNG
58 .NET — Setup & API
59 .NET — NUnit, MSTest & xUnit

OTHER RUNNERS & CROSS-LANGUAGE
60 Library Mode (No Test Runner)
61 Jest, Vitest & BDD
62 Cross-Language API Map

CI & REMOTE EXECUTION
63 GitHub Actions
64 Docker & Other CI
65 Remote & Cloud Browsers

QUICK REFERENCE
66 Quick Reference — Locators, Actions & Assertions
67 Quick Reference — CLI & Config

PW

Table of Contents

67 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 05

01 Introduction & Archite… | 02 Installation | 03 Project Structure
04 CLI Essentials | 05 VS Code Extension

WRITING TESTS 06 – 09

06 Test Structure | 07 Annotations & Tags | 08 Test Steps & Test Info
09 Parameterized Tests

LOCATORS 10 – 13

10 getByRole | 11 Other Built-in Locators | 12 Filtering, Chaining & Ra…
13 Frames, Shadow DOM …

ACTIONS 14 – 17

14 Clicks & Text Input | 15 Forms & Files | 16 Keyboard, Mouse & Drag
17 Dialogs, Downloads & N…

ASSERTIONS 18 – 20

18 Web-First Locator Ass… | 19 Page & Generic Asserti… | 20 Soft, Polling & Custom …

NAVIGATION & WAITING 21 – 22

21 Navigation & Load Stat… | 22 Auto-Waiting & Timeou…

FIXTURES & PAGE OBJECTS 23 – 26

23 Built-in Fixtures | 24 Custom Fixtures | 25 Page Object Model
26 Global Setup & Project …

AUTHENTICATION 27 – 28

27 Auth with storageState | 28 Multi-Role & Advanced…

NETWORK 29 – 31

29 Routing, Mocking & HAR | 30 Waiting on Network Tra… | 31 API Testing

BROWSERS, CONTEXTS & EMULATION 32 – 34

32 Browsers & Channels | 33 Devices & Emulation | 34 Clock, Permissions & St…

VISUAL & ACCESSIBILITY TESTING 35 – 36

35 Visual Comparisons & … | 36 Accessibility & ARIA Sn…

CONFIGURATION & RUNNING TESTS 37 – 41

37 playwright.config.ts | 38 Parallelism & Sharding | 39 Retries & Flaky Tests
40 Reporters | 41 CLI Filtering & Flags

DEBUGGING & TOOLING 42 – 46

42 UI Mode | 43 Trace Viewer | 44 Inspector & Debug Mo…
45 Codegen | 46 Screenshots, Video & S…

AI AGENTS & MCP 47 – 48

47 Test Agents | 48 Playwright MCP & play…

WEB APP FRAMEWORKS 49 – 54

49 webServer & baseURL | 50 React (Vite) Apps | 51 Next.js Apps
52 Vue, Angular & Svelte A… | 53 Component Testing — … | 54 Component Testing — …

LANGUAGE BINDINGS 55 – 59

55 Python — Setup, Sync … | 56 Python — pytest Plugin | 57 Java — Setup, JUnit & T…
58 .NET — Setup & API | 59 .NET — NUnit, MSTest &…

OTHER RUNNERS & CROSS-LANGUAGE 60 – 62

60 Library Mode (No Test … | 61 Jest, Vitest & BDD | 62 Cross-Language API M…

CI & REMOTE EXECUTION 63 – 65

63 GitHub Actions | 64 Docker & Other CI | 65 Remote & Cloud Brows…

QUICK REFERENCE 66 – 67

66 Quick Reference — Loc… | 67 Quick Reference — CLI …

Sheet 01

Introduction & Architecture

Chromium · Firefox · WebKit · browser → context → page · library vs test runner · auto-waiting

WHAT IS PLAYWRIGHT?

Playwright is Microsoft's open-source framework for end-to-end testing and browser automation. One API drives **Chromium**, **Firefox** and **WebKit** in headed or headless mode on Windows, macOS and Linux. It ships two layers: the `playwright` **library** for scripting browsers, and `@playwright/test`, a full **test runner** with fixtures, parallelism, retries, reporters and tracing. Every action **auto-waits** for the element to be ready, which removes most manual sleeps.

KEY FACTS

- Current release: **1.64** (Chromium 156, Firefox 157, WebKit 27.2)
- Official bindings: TypeScript/JavaScript, Python, Java, .NET
- Browsers are patched builds downloaded by `npx playwright install`
- Talks to browsers over CDP / custom protocols, not WebDriver
- Test files: `*.spec.ts` or `*.test.ts` under `testDir`

HELLO, PLAYWRIGHT

```ts
// tests/example.spec.ts
import { test, expect } from '@playwright/test';

test('homepage has a title', async ({ page }) => {   // page fixture = fresh tab
  await page.goto('https://playwright.dev/');          // waits for "load"
  await expect(page).toHaveTitle(/Playwright/);        // retries until true

  await page.getByRole('link', { name: 'Get started' }).click();
  await expect(page.getByRole('heading', { name: 'Installation' }))
    .toBeVisible();                                    // web-first assertion
});
// npx playwright test  → runs in every configured browser, headless
```

CODE EXPLAINED

- → `test` — declares a test; receives fixtures
- → `{ page }` — isolated page in a new browser context
- → `page.goto()` — navigates and waits for load
- → `getByRole()` — user-facing locator (preferred)
- → `expect(...).toBeVisible()` — auto-retrying assertion

OBJECT HIERARCHY

OBJECT	ROLE
`Browser`	One launched browser process
`BrowserContext`	Isolated incognito-like profile (cookies, storage)
`Page`	One tab inside a context
`Frame`	Main frame or iframe of a page
`Locator`	Lazy, re-resolving handle to elements

BROWSER ENGINES

ENGINE	COVERS
`chromium`	Chrome, Edge, Opera, Brave
`firefox`	Firefox (Gecko)
`webkit`	Safari engine (macOS, iOS emulation)
`channel: 'chrome'`	Branded Google Chrome
`channel: 'msedge'`	Branded Microsoft Edge

PACKAGES

PACKAGE	USE
`@playwright/test`	Test runner + library (most projects)
`playwright`	Library only, no runner
`playwright-core`	Library without browser downloads
`@playwright/mcp`	MCP server for AI agents

⚠ COMMON MISTAKES

- Installing `playwright` and hand-rolling a runner when `@playwright/test` already gives fixtures, retries and reports
- Adding `waitForTimeout()` sleeps; actions and `expect` already wait
- Assuming WebKit = real Safari; it is the engine, not the branded browser

✓ TIPS & BEST PRACTICES

- Start with `npm init playwright@latest` ; it scaffolds config, example tests and CI
- Prefer `getByRole` locators; they match how users and assistive tech see the page
- One context per test gives full isolation at almost no cost

QUICK SUMMARY

- `@playwright/test` — runner + assertions + fixtures
- `browser → context → page` — isolation layers
- `auto-wait` — actions wait for actionable elements
- `3 engines` — Chromium, Firefox, WebKit

Installation

INSTALLING PLAYWRIGHT

The quickest start is `npm init playwright@latest`, which installs `@playwright/test`, writes `playwright.config.ts`, adds an example test and optionally a GitHub Actions workflow. Browsers are separate downloads stored in a shared cache; `npx playwright install` fetches the versions matched to your installed Playwright release. On Linux CI, `--with-deps` also installs the OS libraries the browsers need.

REQUIREMENTS

- Node.js 20, 22 or 24 (latest LTS recommended)
- Windows 10+/Server 2016+, macOS 14+ (WebKit needs macOS 15+), Ubuntu 22.04/24.04/26.04, Debian 12/13
- Ubuntu 20.04 and Debian 11 are no longer supported
- Browser cache: `~/.cache/ms-playwright` (Linux), `~/Library/Caches/ms-playwright` (macOS), `%USERPROFILE%\AppData\Local\ms-playwright` (Windows)

INSTALL COMMANDS

```
# New project (interactive: TS/JS, test dir, GitHub Actions, browsers)
npm init playwright@latest
yarn create playwright      # or: pnpm create playwright

# Add to an existing project
npm i -D @playwright/test
npx playwright install                    # all default browsers
npx playwright install chromium           # one browser
npx playwright install --with-deps        # browsers + OS packages (CI)
npx playwright install-deps               # OS packages only
npx playwright install --no-remove        # keep older browser builds
npx playwright install --list             # show installed browsers

# Upgrade
npm i -D @playwright/test@latest && npx playwright install
npx playwright --version                  # Version 1.64.x
```

CODE EXPLAINED

- → `npm init playwright@latest` — scaffold + install in one step
- → `install <browser>` — chromium, firefox, webkit, chrome, msedge
- → `--with-deps` — apt packages for Linux; needs sudo
- → `--no-remove` — don't delete stale browsers on update
- → Re-run `install` after every Playwright upgrade

ENVIRONMENT VARIABLES

VARIABLE	EFFECT
`PLAYWRIGHT_BROWSERS_PATH`	Custom browser cache folder (0 = node_modules)
`PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD`	Skip browser download on install
`PLAYWRIGHT_DOWNLOAD_HOST`	Mirror for browser downloads
`HTTPS_PROXY`	Proxy for browser downloads

WHAT INIT CREATES

FILE	PURPOSE
`playwright.config.ts`	Projects, reporter, use options
`tests/example.spec.ts`	Sample passing test
`tests-examples/`	Larger demo app tests
`.github/workflows/playwright.yml`	Optional CI workflow

⚠ COMMON MISTAKES

- Upgrading `@playwright/test` without re-running `npx playwright install` ; tests fail with "Executable doesn't exist"
- Running `--with-deps` locally on macOS/Windows; it only matters on Linux
- Committing the browser cache or `node_modules/.cache` to git

✓ TIPS & BEST PRACTICES

- Pin the exact version in `package.json` so CI and laptops use the same browsers
- Install only the browsers you test (`install chromium`) to speed up CI
- Use the official Docker image when OS dependencies are painful

QUICK SUMMARY

- `npm init playwright@latest` — scaffold everything
- `npx playwright install` — download matching browsers
- `--with-deps` — add Linux system packages
- `PLAYWRIGHT_BROWSERS_PATH` — move the browser cache

Project Structure

PROJECT LAYOUT

A Playwright project is driven by `playwright.config.ts` at the root. Test files live under `testDir` (default `./tests`) and are matched by `testMatch` (`**/*.@(spec|test).?(c|m)[jt]s?(x)`). Each run writes artifacts (traces, screenshots, videos) to `test-results/` and the HTML report to `playwright-report/`; both are disposable and belong in `.gitignore`.

NOTES

- `testDir` is resolved relative to the config file
- Helpers, fixtures and page objects can live anywhere; only matched files run as tests
- `outputDir` (default `test-results`) is wiped at the start of each run
- `playwright/.auth/` is the conventional home for saved login state

TYPICAL LAYOUT

```
my-app/
├── playwright.config.ts      # runner configuration
├── package.json              # "test:e2e": "playwright test"
├── tests/
│   ├── auth.setup.ts         # setup project (login once)
│   ├── home.spec.ts          # test file
│   ├── checkout/cart.spec.ts # nested folders are fine
│   ├── fixtures.ts           # custom test.extend()
│   └── pages/LoginPage.ts    # page objects (not matched)
├── playwright/.auth/user.json   # saved storageState (gitignored)
├── test-results/             # per-test artifacts (gitignored)
├── playwright-report/        # HTML report (gitignored)
└── blob-report/              # shard reports for merging (gitignored)
```

CODE EXPLAINED

- → `*.spec.ts` — matched by default `testMatch`
- → `*.setup.ts` — matched only when a project's `testMatch` says so
- → `pages/` — plain modules imported by tests
- → `.auth/` — contains session cookies; never commit

RECOMMENDED .GITIGNORE

ENTRY	WHY
`/test-results/`	Regenerated each run
`/playwright-report/`	HTML report output
`/blob-report/`	Shard reports
`/playwright/.cache/`	Component-testing build cache
`/playwright/.auth/`	Logged-in session secrets

PACKAGE.JSON SCRIPTS

SCRIPT	COMMAND
`test:e2e`	playwright test
`test:ui`	playwright test --ui
`test:headed`	playwright test --headed
`report`	playwright show-report
`codegen`	playwright codegen localhost:3000

⚠ COMMON MISTAKES

- Putting page objects in files ending `.spec.ts`; they get collected and run as empty tests
- Committing `playwright/.auth/*.json`; it holds live session cookies
- Writing files into `test-results/` and expecting them later; the folder is cleared per run

✓ TIPS & BEST PRACTICES

- Use `testInfo.outputPath()` for per-test scratch files
- Keep e2e tests in their own folder so unit-test runners (Vitest/Jest) ignore them
- Add `testIgnore` for folders the runner should skip

QUICK SUMMARY

- `playwright.config.ts` — single source of configuration
- `testDir` + `testMatch` — which files are tests
- `test-results/` — artifacts, cleared per run
- `playwright-report/` — HTML report

CLI Essentials

THE PLAYWRIGHT CLI

Everything runs through `npx playwright <command>`. `test` runs the suite and accepts file filters, line numbers and many flags; `show-report` opens the HTML report; `codegen` records interactions into code; `show-trace` and the newer `trace` subcommands inspect recorded traces. The CLI also bundles the MCP server (`playwright mcp`) and the agent CLI (`playwright cli`).

NOTES

- Positional args to `test` are regex filters on file paths
- `file.spec.ts:42` runs only the test at line 42
- Flags override matching config values for that run
- `npx playwright --help` / `<command> --help` lists everything

EVERYDAY COMMANDS

```
npx playwright test                          # whole suite, all projects
npx playwright test tests/login.spec.ts      # one file
npx playwright test login cart               # files matching "login" or "cart"
npx playwright test login.spec.ts:18         # test at line 18
npx playwright test --project=chromium       # one project
npx playwright test --headed --workers=1     # watch it, serially
npx playwright test --ui                     # interactive UI mode
npx playwright test --debug                  # Inspector, step through
npx playwright test -g "add to cart"         # filter by title
npx playwright show-report                   # open HTML report
npx playwright codegen https://example.com   # record a test
npx playwright show-trace trace.zip          # open a trace
```

CODE EXPLAINED

- → `test [filters]` — runs matching files
- → `:line` — pins to one test or describe
- → `--project` — repeatable; matches project `name`
- → `--ui` / `--debug` — interactive modes
- → `show-report` — serves `playwright-report/`

COMMANDS

COMMAND	PURPOSE
`test`	Run tests
`show-report [dir]`	Serve an HTML report
`codegen [url]`	Record actions → code
`show-trace / trace`	Inspect trace files
`install / install-deps`	Browsers and OS deps
`merge-reports`	Combine shard blob reports
`init-agents / init-skills`	Scaffold AI agent files
`mcp / cli`	Bundled MCP server / agent CLI

COMMON TEST FLAGS

FLAG	EFFECT
`--headed`	Show browser windows
`--workers=N`	Parallel worker count
`--retries=N`	Retry failures
`--reporter=list`	Override reporter
`--update-snapshots`	Rewrite baselines
`--last-failed`	Re-run only failures
`--list`	List tests, don't run
`--timeout=ms`	Per-test timeout

⚠ COMMON MISTAKES

- Passing a test title as a positional argument; positionals filter **file paths**, use `-g` for titles
- Using `--headed` with many workers and wondering why the machine crawls
- Forgetting `--` when running through npm: `npm test -- --headed`

✓ TIPS & BEST PRACTICES

- `--last-failed` after a red run is the fastest feedback loop
- `--list` confirms which tests a filter selects before running them
- Add common invocations as npm scripts so the team uses the same flags

QUICK SUMMARY

- `test` — run, filter by path, line, project, title
- `show-report` — view results
- `codegen` — record new tests
- `show-trace` — post-mortem debugging

VS Code Extension

Test Explorer · run/debug gutter · Pick locator · Record new · Show browser · trace in editor

PLAYWRIGHT TEST FOR VS CODE

The official **Playwright Test for VS Code** extension (ms-playwright.playwright) integrates the runner into the editor. Tests appear in the **Test Explorer** and get run/debug icons in the gutter. A Playwright sidebar lets you choose projects, toggle **Show browser**, **Pick locator** from a live page, **Record new** tests with codegen, and open traces without leaving the editor.

NOTES

- Needs `@playwright/test` installed in the workspace
- Detects multiple config files; pick the active one in the sidebar
- Breakpoints work in tests and page objects (debug mode)
- Run the "Install Playwright" command to scaffold a new project

SIDEBAR & GUTTER FEATURES

FEATURE	WHAT IT DOES
`Run / Debug gutter icon`	Run one test, describe or file; debug honours breakpoints
`Test Explorer`	Tree of all tests, filter, run all, rerun failed
`Projects checkboxes`	Choose which config projects run
`Show browser`	Headed runs that reuse one browser window
`Pick locator`	Hover the page, copy the best locator into the editor
`Record new`	Opens codegen; generated code lands in a new spec file
`Record at cursor`	Appends recorded steps into the current test
`Show trace viewer`	Opens the trace for the last run inside VS Code
`Reveal test output`	Shows stdout/stderr and failure details

WORKFLOW

```
test('checkout', async ({ page }) => {
  await page.goto('/cart');
  // 1. put cursor here → "Record at cursor"
  // 2. click through the app; steps are typed in live
  await page.getByRole('button', { name: 'Checkout' }).click();
  // 3. set a breakpoint on the next line → Debug test
  await expect(page.getByText('Order placed')).toBeVisible();
});
// 4. hover a locator in the editor → it highlights in the open browser
```

CODE EXPLAINED

- → Record at cursor — fastest way to extend a test
- → Breakpoints — pause with full variable inspection
- → Live highlight — editing a locator re-highlights matches
- → Show browser — keeps one window open between runs

⚠ COMMON MISTAKES

- Opening the folder above the project root; the extension can't find the config
- Leaving "Show browser" on in parallel runs; everything funnels through one window
- Expecting codegen output to be final; rename and restructure recorded steps

✓ TIPS & BEST PRACTICES

- Use "Pick locator" to settle locator debates quickly
- Run a single test from the gutter instead of the whole file while iterating
- Combine with UI mode (`--ui`) for watch mode and timeline debugging

QUICK SUMMARY

- `gutter icons` — run/debug one test
- `Pick locator` — copy a robust locator
- `Record new / at cursor` — codegen in editor
- `Show trace viewer` — debug in VS Code

Test Structure

ANATOMY OF A TEST FILE

A test file imports `test` and `expect` from `@playwright/test`. Each `test(title, async ({ fixtures }) => {})` gets fresh fixtures, so every test has its own **browser context** and **page**. Group tests with `test.describe()` and share setup with **hooks**. Tests in a file run in order inside one worker; files run in parallel across workers.

RULES OF THUMB

- Every test is fully isolated: new context, new cookies, new storage
- `beforeAll` / `afterAll` run once **per worker**, not once per run
- Hooks can use fixtures: `beforeEach(async ({ page }) => …)`
- Titles must be unique within a file (used in reports and `-g`)

FULL EXAMPLE

```
import { test, expect } from '@playwright/test';

test.describe('todo list', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/todos');                  // runs before each test
  });

  test('adds an item', async ({ page }) => {
    await page.getByPlaceholder('What needs doing?').fill('Buy milk');
    await page.keyboard.press('Enter');
    await expect(page.getByTestId('todo-item')).toHaveText(['Buy milk']);
  });

  test('starts empty', async ({ page }) => {
    await expect(page.getByTestId('todo-item')).toHaveCount(0);
  });

});
```

CODE EXPLAINED

- → `test.describe()` — named group; nests freely
- → `beforeEach` — shared navigation per test
- → `{ page }` — destructure only the fixtures you need
- → `testInfo` — 2nd argument to tests and hooks

HOOKS

HOOK	RUNS
`test.beforeAll`	Once per worker before tests in scope
`test.beforeEach`	Before every test in scope
`test.afterEach`	After every test, pass or fail
`test.afterAll`	Once per worker after tests in scope

DECLARATION FORMS

FORM	MEANING
`test(title, fn)`	Basic test
`test(title, details, fn)`	With `{ tag, annotation }`
`test.describe(title, fn)`	Named group
`test.describe(fn)`	Anonymous group (shared hooks)
`test.describe.serial`	Ordered; stop after first failure

⚠ COMMON MISTAKES

- Sharing a `page` between tests via a module variable; each test must use its own fixture
- Using `beforeAll` to log in for every test, then wondering why parallel workers each log in again
- Forgetting `await` on actions; the test ends before the action runs and fails mysteriously

✓ TIPS & BEST PRACTICES

- Keep tests independent so they can run in any order and in parallel
- Put navigation in `beforeEach`, not in every test body
- Prefer fixtures over `beforeAll` for shared expensive resources

QUICK SUMMARY

- `test()` — one isolated scenario
- `test.describe()` — grouping + shared hooks
- `beforeEach/afterEach` — per-test setup/teardown
- `beforeAll/afterAll` — per-worker setup/teardown

Annotations & Tags

ANNOTATIONS AND TAGS

Annotations change how a test is treated: skipped, expected to fail, given more time, or focused. They can be static (`test.skip('title', …)`) or conditional inside the test (`test.skip(browserName === 'webkit', 'reason')`). **Tags** are labels such as `@smoke` that you filter on with `--grep`. Both appear in the HTML report.

NOTES

- Tags must start with `@`
- Tags can also be written in the title: `'login @smoke'`
- `test.only` + `forbidOnly: !!process.env.CI` blocks accidental commits
- `test.slow()` triples the timeout

EXAMPLES

```ts
test('checkout', { tag: ['@smoke', '@payments'] }, async ({ page }) => {});

test.describe('admin', { tag: '@admin' }, () => {
  test('bulk delete', async ({ page }) => { /* inherits @admin */ });
});

test('mobile menu', async ({ page, isMobile }) => {
  test.skip(!isMobile, 'Only relevant on mobile viewports');
  test.slow();                                    // 3× timeout
});

test.fixme('known broken export', async ({ page }) => { /* not run */ });
test.fail('bug #412 still open', async ({ page }) => { /* must fail */ });

test('issue link', {
  annotation: { type: 'issue', description: 'https://github.com/org/app/issues/9' },
}, async ({ page }) => { /* shown in report */ });
```

CODE EXPLAINED

- → `{ tag }` — string or array; inherited by nested tests
- → `test.skip(cond, why)` — runtime skip
- → `test.fixme` — skip and mark as needing work
- → `test.fail` — passes only if it fails
- → `annotation` — custom `{ type, description }`

ANNOTATIONS

CALL	EFFECT
`test.skip()`	Not run; reported as skipped
`test.fixme()`	Not run; marked "fixme"
`test.fail()`	Inverts pass/fail
`test.slow()`	Timeout × 3
`test.only()`	Run only focused tests

FILTERING BY TAG

COMMAND	SELECTS
`--grep @smoke`	Tests tagged @smoke
`-g "@smoke|@fast"`	Either tag (regex)
`--grep-invert @slow`	Everything except @slow
`-G @slow`	Short form of --grep-invert
`--grep "(?=.*@a)(?=.*@b)"`	Both tags

⚠ COMMON MISTAKES

- Committing `test.only` ; CI runs one test and goes green
- Using `test.fail()` for flaky tests; it hides real regressions when the bug is fixed
- Writing tags without `@` ; Playwright rejects them

✓ TIPS & BEST PRACTICES

- Set `forbidOnly: !!process.env.CI` in config
- Always give `skip` / `fixme` a reason string so the report explains itself
- Use project-level `grep` to make a "smoke" project that runs only `@smoke`

QUICK SUMMARY

- `skip` / `fixme` — don't run
- `fail` — expect failure
- `slow` — 3× timeout
- `{ tag: '@x' }` + `--grep` — label and filter

Test Steps & Test Info

STEPS AND TESTINFO

`test.step()` groups actions into named steps that show up in the HTML report, trace viewer and UI mode, turning a long test into a readable story. Steps can return values and nest. `testInfo` (or `test.info()`) exposes the running test's metadata and lets you attach files, add annotations and get per-test output paths.

NOTES

- `box: true` reports failures at the step call site, not inside it
- `subtitle` and `params` (1.63) add detail in reports/trace
- `timeout` option fails a step that hangs
- `test.abort(message?)` fails the test immediately (fixtures, hooks, route handlers)

STEPS, ATTACHMENTS, ABORT

```ts
test('purchase', async ({ page }) => {
  const orderId = await test.step('place order', async () => {
    await page.getByRole('button', { name: 'Buy' }).click();
    return page.getByTestId('order-id').textContent();   // step returns a value
  });

  await test.step('verify email', async () => {
    /* … */
  }, { box: true, timeout: 10_000, subtitle: `order ${orderId}` });

  const info = test.info();
  await info.attach('order', { body: JSON.stringify({ orderId }),
                               contentType: 'application/json' });
  await page.screenshot({ path: info.outputPath('s.png') });
  await info.attach('screen', { path: info.outputPath('s.png') });
  info.annotations.push({ type: 'order', description: String(orderId) });
});

// in a route handler or fixture:
// if (!process.env.API_KEY) test.abort('API_KEY not set');
```

CODE EXPLAINED

- → `test.step(title, fn, opts)` — named, nestable
- → `return` — value flows out of the step
- → `attach(name, { body | path })` — shows in report
- → `outputPath()` — unique per-test file path
- → `annotations.push()` — runtime annotation

USEFUL TESTINFO FIELDS

FIELD	CONTAINS
`title / titlePath`	Test name / full path
`status / expectedStatus`	Result vs expected
`retry`	0 on first run, 1+ on retries
`project`	Current project config
`outputDir`	This test's artifact folder
`timeout / setTimeout()`	Read / change the test timeout

STEP OPTIONS

OPTION	EFFECT
`box`	Errors point at the step call
`timeout`	Fail if the step runs longer
`subtitle`	Extra text next to the title
`params`	Values shown in trace viewer
`test.step.skip`	Declare a skipped step

⚠ COMMON MISTAKES

- Calling `test.info()` at module level; it throws outside a running test
- Attaching huge files on every run; attach only on failure or when useful
- Writing artifacts to fixed paths; parallel tests overwrite each other, use `outputPath()`

✓ TIPS & BEST PRACTICES

- Wrap page-object methods in `test.step` so reports read like specs
- Use `box: true` on helper steps so errors point at the caller
- Check `testInfo.retry > 0` to add extra diagnostics only on retries

QUICK SUMMARY

- `test.step()` — readable, reportable steps
- `testInfo.attach()` — files in the report
- `outputPath()` — collision-free artifact paths
- `test.abort()` — fail fast from fixtures/handlers

Parameterized Tests

DATA-DRIVEN TESTS

Playwright has no special parameterize API: you generate tests with plain loops at declaration time. Each generated test needs a **unique title**. For values that should vary per run (locale, user role, environment), declare a fixture **option** and set it per project or with `test.use()`, so the same test runs under several configurations.

NOTES

- Loops must run at module load, not inside a test
- Data can come from JSON/CSV read synchronously at the top of the file
- `test.use()` applies to a file or describe block
- Parallel mode lets generated tests in one file run concurrently

LOOP + DATA FILE

```ts
import { test, expect } from '@playwright/test';
import users from './data/users.json';            // [{ name, role, canDelete }]

test.describe.configure({ mode: 'parallel' });

for (const { name, role, canDelete } of users) {
  test(`${role} sees correct actions (${name})`, async ({ page }) => {
    await page.goto(`/login-as/${name}`);
    const del = page.getByRole('button', { name: 'Delete' });
    if (canDelete) await expect(del).toBeVisible();
    else await expect(del).toBeHidden();
  });
}

test.describe('German UI', () => {
  test.use({ locale: 'de-DE', timezoneId: 'Europe/Berlin' });
  test('greeting', async ({ page }) => { /* runs with de-DE */ });
});
```

CODE EXPLAINED

- → `for…of` — one test per row
- → Template literal title — keeps titles unique
- → `mode: 'parallel'` — rows run concurrently
- → `test.use()` — override options for a block

PROJECT-LEVEL PARAMETER

```ts
// fixtures.ts
export const test = base.extend<{ tier: 'free' | 'pro' }>({
  tier: ['free', { option: true }],
});
// playwright.config.ts
projects: [
  { name: 'free', use: { tier: 'free' } },
  { name: 'pro',  use: { tier: 'pro' } },
],
```

CODE EXPLAINED

- → `{ option: true }` — settable from config
- → Each project runs the whole suite with its value
- → Tests read it like any fixture: `({ tier })`

⚠ COMMON MISTAKES

- Building tests inside `beforeAll` or an async function; tests must be declared synchronously
- Duplicate titles from loops; Playwright errors on duplicates
- Reading data with `await fetch()` at the top of the file; declaration must be synchronous

✓ TIPS & BEST PRACTICES

- Put the parameter in the title so failures are self-explanatory
- Use options + projects for cross-cutting variants (roles, tiers, locales)
- Use `test.describe` per data row when each row has several tests

QUICK SUMMARY

- `for…of` — generate tests from data
- `test.use()` — override options per block
- `{ option: true }` — project-configurable fixture
- `mode: 'parallel'` — run rows concurrently

getByRole

ARIA roles · name · exact · checked / pressed / expanded · level · description

LOCATING BY ROLE

`getByRole()` is Playwright's **recommended** locator. It finds elements the way users and assistive technology perceive them: by ARIA role (implicit or explicit) and **accessible name**. Because it ignores CSS classes and DOM structure, it survives refactors, and a failing role locator often points to a real accessibility bug.

NOTES

- Name match is case-insensitive substring by default
- `exact: true` requires a full, case-sensitive match
- `name` accepts a RegExp: `{ name: /sign (in|up)/i }`
- Hidden elements are excluded unless `includeHidden: true`

EXAMPLES

```
page.getByRole('button', { name: 'Submit' })                  // <button>Submit</button>
page.getByRole('link', { name: 'Docs', exact: true })         // not "API Docs"
page.getByRole('heading', { level: 2, name: /pricing/i })     // <h2>
page.getByRole('checkbox', { name: 'Remember me', checked: true })
page.getByRole('button', { name: 'Bold', pressed: true })     // aria-pressed
page.getByRole('button', { name: 'Menu', expanded: false })   // aria-expanded
page.getByRole('option', { name: 'Blue', selected: true })
page.getByRole('textbox', { name: 'Email' })                  // via <label>
page.getByRole('row', { name: /Alice/ }).getByRole('button', { name: 'Edit' })
page.getByRole('img', { name: 'Logo', description: 'Company logo' })   // 1.60
page.getByRole('dialog').getByRole('button', { name: 'Close' })
```

CODE EXPLAINED

- → role — first argument, ARIA role string
- → `name` — accessible name (text, label, aria-label)
- → state filters — `checked`, `pressed`, `expanded`, `selected`, `disabled`
- → `level` — heading level 1–6
- → chain — scope to a row, dialog or region

IMPLICIT ROLES

ELEMENT	ROLE
`<button>, input[type=submit]`	button
`<a href>`	link
`<input type=text>, textarea`	textbox
`input[type=checkbox] / radio`	checkbox / radio
`<select> / <option>`	combobox / option
`<h1>-<h6>`	heading
`<ul>,<ol> / <li>`	list / listitem
`<nav> / <main>`	navigation / main

OPTIONS

OPTION	TYPE
`name`	string | RegExp
`exact`	boolean
`checked / pressed`	boolean ('mixed' for checked)
`expanded / selected`	boolean
`disabled`	boolean
`level`	number
`includeHidden`	boolean
`description`	string | RegExp

⚠ COMMON MISTAKES

- Using the visible text of an icon button; its accessible name comes from `aria-label`
- Expecting `name: 'Save'` to exclude "Save draft"; substring match needs `exact: true`
- Adding redundant `role="button"` to a `<button>`; implicit roles already match

✓ TIPS & BEST PRACTICES

- Use the Pick Locator tool to see the role and name Playwright computes
- When no role locator works, fix the markup (labels, `aria-label`) rather than falling back to CSS
- Scope by landmark (`dialog`, `navigation`, `row`) to keep names short

QUICK SUMMARY

- `getByRole(role, { name })` — preferred locator
- `exact` — full-string match
- state options — filter by ARIA state
- chaining — scope inside containers

Other Built-in Locators

THE GETBY* FAMILY

Besides `getByRole`, Playwright ships six user-facing locators. Use **getByLabel** for form fields, **getByText** for non-interactive content, **getByPlaceholder**/**getByAltText**/**getByTitle** when those attributes are the best handle, and **getByTestId** when you control the markup and need a stable contract that is independent of copy and language.

MATCHING RULES

- Strings: case-insensitive substring, whitespace normalized
- `{ exact: true }` : case-sensitive, full string
- RegExp: `getByText(/welcome, \w+/i)`
- `getByTestId` is always exact; default attribute `data-testid`

EXAMPLES

```
page.getByLabel('Email address')                  // <label>, aria-label(ledby)
page.getByLabel('Password', { exact: true })      // not "Confirm password"
page.getByPlaceholder('name@example.com')
page.getByText('Welcome back')                    // substring, any case
page.getByText('Total', { exact: true })          // only the exact node text
page.getByText(/order #\d+ confirmed/i)
page.getByAltText('Company logo')                 // <img alt>
page.getByTitle('Issues count')                   // title="…"
page.getByTestId('cart-badge')                    // data-testid="cart-badge"

// playwright.config.ts — use a custom attribute
export default defineConfig({ use: { testIdAttribute: 'data-qa' } });
// or at runtime: selectors.setTestIdAttribute('data-qa');
```

CODE EXPLAINED

- → `getByLabel` — best for inputs, selects, checkboxes
- → `getByText` — matches the smallest element containing the text
- → `exact` — opt out of substring matching
- → `testIdAttribute` — rename `data-testid`

WHICH LOCATOR?

SITUATION	USE
Button, link, heading, tab	getByRole
Form field with a label	getByLabel
Field with no label	getByPlaceholder
Paragraph / message text	getByText
Image	getByAltText
Copy changes or i18n	getByTestId

RECOMMENDED PRIORITY

RANK	LOCATOR
1	getByRole
2	getByLabel
3	getByPlaceholder
4	getByText
5	getByAltText / getByTitle
6	getByTestId
7	CSS / XPath (last resort)

⚠ COMMON MISTAKES

- `getByText('Save')` on a page with "Save" and "Save draft"; strict mode throws, use `exact` or a role locator
- Using `getByText` for buttons; `getByRole('button', { name })` is stricter and accessible
- Forgetting that `getByTestId` is exact while other getBy* are substring by default

✓ TIPS & BEST PRACTICES

- Agree on `data-testid` naming (`feature-element`) with developers
- Text locators break on copy changes and translations; prefer roles or labels for i18n apps
- Run codegen to see which locator Playwright would pick

QUICK SUMMARY

- `getByLabel` — form controls
- `getByText` — visible content
- `getByTestId` — stable dev contract
- `testIdAttribute` — custom test-id attribute

Sheet 12

Filtering, Chaining & Raw Locators

filter({ hasText, has, hasNot }) · and() / or() · nth() / first() / last() · within() · visible() · locator() CSS / XPath · >> chaining

NARROWING LOCATORS

Locators are **lazy** and **chainable**: calling a getBy method on a locator searches inside it. `filter()` keeps matches that contain (or don't contain) text or another locator. `and()`/`or()` combine locators, `within()` (1.64) restricts one locator to another container, and `visible()` keeps only visible matches. Raw CSS and XPath via `page.locator()` remain available as a last resort.

NOTES

- Strictness: actions throw if a locator matches more than one element
- `nth()` / `first()` / `last()` silence strictness but are brittle
- XPath does not pierce shadow DOM; CSS and getBy* do
- `has` locators are relative to the outer element

EXAMPLES

```
const rows = page.getByRole('row');
rows.filter({ hasText: 'Alice' })  // row containing "Alice"
rows.filter({ hasNotText: 'Archived' })
rows.filter({ has: page.getByRole('button', { name: 'Edit' }) })  // contains a button
rows.filter({ hasNot: page.getByText('Paid') })
rows.filter({ hasText: 'Alice' }).getByRole('button', { name: 'Delete' }).click();

page.getByRole('button').and(page.getByTitle('Subscribe'))  // both match
page.getByRole('button', { name: 'New' }).or(page.getByText('Confirm'))  // either
page.getByRole('button', { name: 'Save' }).within(page.getByTestId('settings-dialog'))
page.locator('button').visible().click();  // visible only
rows.first(); rows.last(); rows.nth(2);  // index (0-based)

page.locator('css=.card >> text=Pro')  // legacy chained selector
page.locator('#cart .item:has-text("Mug")')  // CSS + Playwright pseudo-class
page.locator('xpath=//table//tr[2]/td[1]')  // XPath (no shadow DOM)
```

CODE EXPLAINED

- → `filter()` — narrows by text or inner locator
- → `.getBy…()` on a locator — searches inside it
- → `and()` / `or()` — intersect / union
- → `within()` — scope to a container (1.64)
- → `nth()` — positional, use sparingly
- → `locator()` — raw CSS/XPath escape hatch

FILTER() OPTIONS

OPTION	KEEPS ELEMENTS THAT
`hasText`	Contain the text / RegExp
`hasNotText`	Do not contain the text
`has`	Contain a match of the locator
`hasNot`	Do not contain a match
`visible`	Are visible (true) / hidden (false)

CSS EXTENSIONS

SELECTOR	MATCHES
`:has-text("x")`	Element containing text
`:text("x")`	Smallest element with text
`:visible`	Visible elements (prefer visible())
`:nth-match(sel, 2)`	2nd match overall
`>>`	Chain selectors (legacy)

⚠ COMMON MISTAKES

- `rows.nth(3)` to find "Alice's row"; order changes break it, filter by content instead
- Passing a page-level locator to `has` that can't be found inside the outer element
- Copying long auto-generated CSS paths (`div > div:nth-child(4) > span`) from DevTools

✓ TIPS & BEST PRACTICES

- Build locators top-down: container → filter → target role
- Store reusable container locators in page objects
- Use `or()` for "either the dialog or the banner appears" races

QUICK SUMMARY

- `filter({ hasText, has })` — narrow by content
- `and / or` — combine locators
- `within()` — restrict to a container
- `locator('css')` — last-resort raw selector

Sheet 13

Frames, Shadow DOM & Lists

frameLocator() · contentFrame() · shadow piercing · count() · all() · strictness errors

FRAMES, SHADOW DOM AND LISTS

Content inside an <iframe> needs a **FrameLocator**: `page.frameLocator(selector)` or `locator.contentFrame()`. Since 1.63, `page.frameLocator()` searches across all frames. Open **shadow DOM** is pierced automatically by every locator except XPath. When a locator matches many elements, assert with `toHaveCount/toHaveText([...])`, or iterate with `all()`.

NOTES

- Closed shadow roots are not supported
- `count()` and `all()` do not wait; assert first, then iterate
- Strict mode error lists every match — read it to fix the locator
- `page.frames()` / `page.frame({ url })` give Frame objects

EXAMPLES

```
// iframes
const stripe = page.frameLocator('iframe[title="Secure card input"]');
await stripe.getByPlaceholder('Card number').fill('4242 4242 4242 4242');
const map = page.getByTitle('Embedded map').contentFrame();
await map.getByRole('button', { name: 'Zoom in' }).click();

// shadow DOM: write locators as if the shadow root weren't there
await page.getByRole('button', { name: 'Details' }).click(); // in <x-details>

// lists
const items = page.getByRole('listitem');
await expect(items).toHaveCount(3);                    // waits
await expect(items).toHaveText(['Apple', 'Banana', 'Cherry']);
for (const item of await items.all()) {                // no waiting!
  console.log(await item.textContent());
}
const names = await items.allTextContents();           // string[]
```

CODE EXPLAINED

- → `frameLocator()` — locate inside an iframe
- → `contentFrame()` — locator → frame locator
- → shadow DOM — pierced automatically
- → `toHaveCount` / `toHaveText([])` — wait for list state
- → `all()` — snapshot array of locators

LIST APIS

API	WAITS?
`expect(l).toHaveCount(n)`	Yes
`expect(l).toHaveText([...])`	Yes
`l.count()`	No
`l.all()`	No
`l.allTextContents()`	No
`l.allInnerTexts()`	No

FRAME APIS

API	RETURNS
`page.frameLocator(sel)`	FrameLocator
`locator.contentFrame()`	FrameLocator
`frameLocator.owner()`	Locator for the iframe
`page.frame({ name | url })`	Frame | null
`page.mainFrame()`	Frame

⚠ COMMON MISTAKES

- Calling `page.getByText()` for text inside an iframe; it never matches, use a FrameLocator
- Looping over `await items.all()` before the list has loaded; it returns `[]` silently
- Using XPath for shadow-DOM components; XPath does not pierce shadow roots

✓ TIPS & BEST PRACTICES

- Assert `toHaveCount` before `all()` so the list is ready
- Prefer `toHaveText([...])` over manual loops for list checks
- For cross-origin iframes (payments) target by `title` attribute

QUICK SUMMARY

- `frameLocator()` — enter an iframe
- shadow DOM — transparent to locators
- `toHaveCount / toHaveText([])` — list assertions
- `all()` — iterate after waiting

Clicks & Text Input

click() · dblclick() · fill() · pressSequentially() · clear() · force · position · modifiers

CLICKING AND TYPING

Actions live on locators. Before each action Playwright runs **actionability checks** (attached, visible, stable, enabled, receives events) and waits until they pass. `fill()` sets an input's value in one go and fires `input`; use `pressSequentially()` only when the page reacts to individual keystrokes (autocomplete, masks).

NOTES

- `fill('')` or `clear()` empties a field
- `force: true` skips actionability checks (hides real bugs)
- `trial: true` runs the checks without acting
- `click()`'s `noWaitAfter` is deprecated

EXAMPLES

```
const btn = page.getByRole('button', { name: 'Save' });
await btn.click();                                  // left click
await btn.dblclick();
await btn.click({ button: 'right' });               // context menu
await btn.click({ modifiers: ['Shift'] });          // Alt, Control, Meta…
await btn.click({ position: { x: 5, y: 5 } });      // offset from top-left
await btn.click({ clickCount: 3 });                 // triple-click
await btn.click({ trial: true });                   // checks only
await btn.click({ force: true });                   // skip checks

const email = page.getByLabel('Email');
await email.fill('ada@example.com');                // replace value
await email.clear();
await page.getByLabel('Search').pressSequentially('play', { delay: 80 });
await email.press('Enter');
await page.getByLabel('Bio').fill('line 1\nline 2'); // textarea
```

CODE EXPLAINED

- → `click(opts)` — button, modifiers, position, clickCount
- → `fill()` — fastest, fires one input event
- → `pressSequentially()` — per-key events, optional delay
- → `press()` — single key or chord
- → `force` — last resort for covered elements

ACTIONABILITY CHECKS

CHECK	MEANING
`Attached`	Element is in the DOM
`Visible`	Non-empty box, not visibility:hidden
`Stable`	Not animating (2 frames)
`Enabled`	Not disabled
`Receives Events`	Not covered by another element
`Editable`	Not readonly (fill/clear)

COMMON OPTIONS

OPTION	EFFECT
`timeout`	Per-action timeout (ms)
`signal`	AbortSignal to cancel (1.62)
`force`	Skip actionability
`trial`	Checks only, no action
`delay`	ms between mousedown/up or keys
`steps`	Intermediate mouse moves

⚠ COMMON MISTAKES

- `type()` in new code; it's deprecated, use `fill()` or `pressSequentially()`
- `force: true` to "fix" a click intercepted by an overlay; close the overlay instead
- Checking a value right after `pressSequentially` without `expect`; debounced UIs update later

✓ TIPS & BEST PRACTICES

- Read "element intercepts pointer events" errors: they name the covering element
- `ControlOrMeta` writes cross-platform shortcuts (Cmd on macOS)
- Use `fill()` by default; it's faster and less flaky

QUICK SUMMARY

- `click()` — waits, then clicks
- `fill()` — set value instantly
- `pressSequentially()` — real keystrokes
- `force / trial` — skip / dry-run checks

Forms & Files

FORM CONTROLS AND FILE UPLOADS

Checkboxes and radios use `check()`/`uncheck()`/`setChecked()`, which verify the state actually changed. `<select>` elements use `selectOption()` by value, label or index. File inputs accept paths or in-memory buffers through `setInputFiles()`. For drop zones without an `<input>`, use `locator.drop()` (1.60) or catch the `filechooser` event.

NOTES

- `selectOption` returns the selected values
- `setInputFiles([])` clears the selection
- Custom (non-native) dropdowns: click to open, then `getByRole('option')`
- Paths are relative to the current working directory

EXAMPLES

```
await page.getByLabel('I agree').check();
await page.getByLabel('Newsletter').uncheck();
await page.getByLabel('Dark mode').setChecked(true);
await page.getByLabel('Express').check();                  // radio

const size = page.getByLabel('Size');
await size.selectOption('m');                              // by value
await size.selectOption({ label: 'Large' });               // by visible label
await size.selectOption({ index: 0 });
await page.getByLabel('Tags').selectOption(['a', 'b']);    // multi-select

const file = page.getByLabel('Avatar');
await file.setInputFiles('fixtures/avatar.png');
await file.setInputFiles(['a.pdf', 'b.pdf']);
await file.setInputFiles({ name: 'n.txt', mimeType: 'text/plain',
                           buffer: Buffer.from('hi') });           // in memory
await file.setInputFiles([]);                              // clear

const chooser = page.waitForEvent('filechooser');          // no <input>
await page.getByRole('button', { name: 'Upload' }).click();
await (await chooser).setFiles('report.csv');
await page.getByTestId('dropzone').drop({ files: 'photo.jpg' });   // 1.60
```

CODE EXPLAINED

- → `check()` — asserts the box becomes checked
- → `selectOption()` — value, `{ label }`, `{ index }`
- → `setInputFiles()` — paths, buffers, or `[]`
- → `filechooser` — start waiting before clicking
- → `drop()` — simulate an external file drop

MATCHING ASSERTIONS

ASSERTION	CHECKS
`toBeChecked()`	Checkbox / radio checked
`toBeChecked({ checked: false })`	Unchecked
`toHaveValue('m')`	Input / select value
`toHaveValues(['a','b'])`	Multi-select values
`toBeEditable()`	Not readonly

CUSTOM DROPDOWN PATTERN

STEP	CODE
Open	getByRole('combobox').click()
Pick	getByRole('option', { name }).click()
Verify	expect(combobox).toHaveText(name)

⚠ COMMON MISTAKES

- `click()` on a checkbox that toggles twice via label + input; `check()` is idempotent
- `selectOption` on a styled div dropdown; it only works on native `<select>`
- Awaiting the `filechooser` promise **before** clicking; it never resolves

✓ TIPS & BEST PRACTICES

- Keep upload fixtures in a `fixtures/` folder and resolve with `path.join(__dirname, …)`
- Use in-memory buffers for generated files (CSV, JSON) to avoid temp files
- Assert with `toHaveValue` after `selectOption` to catch JS-driven resets

QUICK SUMMARY

- `check/uncheck/setChecked` — toggles with verification
- `selectOption()` — native selects
- `setInputFiles()` — file inputs
- `drop()` / `filechooser` — other upload UIs

Keyboard, Mouse & Drag

press() · keyboard.down/up · mouse.move/wheel · hover() · dragTo() · focus() · scroll option

LOW-LEVEL INPUT

Most tests only need locator actions, but `page.keyboard` and `page.mouse` give raw control for shortcuts, canvas apps, sliders and custom drag-and-drop. `locator.press()` focuses the element first; `page.keyboard.press()` sends to whatever is focused. `dragTo()` handles standard HTML5 and pointer-based drag-and-drop.

KEY NAMES

- Letters/digits: `a`, `A`, `1`; named: `Enter`, `Tab`, `Escape`, `Backspace`
- Arrows: `ArrowUp` … `ArrowRight`; `Home`, `End`, `PageDown`, `F1` – `F12`
- Chords with `+`: `Shift+Tab`, `Control+A`
- `ControlOrMeta` → Cmd on macOS, Ctrl elsewhere

EXAMPLES

```
await page.getByLabel('Search').press('Enter');
await page.getByRole('textbox').press('ControlOrMeta+A');   // select all
await page.keyboard.press('Shift+Tab');                     // focused element
await page.keyboard.down('Shift');                          // hold
await page.keyboard.press('ArrowDown');
await page.keyboard.up('Shift');
await page.keyboard.insertText('héllo 👋');                  // no key events
await page.getByRole('menuitem', { name: 'File' }).hover();
await page.getByLabel('Name').focus();

await page.mouse.move(100, 200);                            // canvas drawing
await page.mouse.down();
await page.mouse.move(300, 200, { steps: 10 });
await page.mouse.up();
await page.mouse.wheel(0, 600);                             // scroll down

await page.getByText('Task A').dragTo(page.getByTestId('done-column'));
await page.getByRole('slider').click({ position: { x: 40, y: 5 } });
```

CODE EXPLAINED

- → `locator.press()` — focus + key
- → `keyboard.down/up` — hold modifiers
- → `insertText()` — paste-like, no keydown
- → `mouse.move(x, y, { steps })` — smooth drag path
- → `dragTo()` — drag source to target

PAGE.KEYBOARD

METHOD	USE
`press(key)`	Down + up
`down(key) / up(key)`	Hold / release
`type(text)`	Per-char key events
`insertText(text)`	Insert without key events

PAGE.MOUSE

METHOD	USE
`move(x, y, opts)`	Move pointer
`down() / up()`	Press / release
`click(x, y)`	Click at coordinates
`wheel(dx, dy)`	Scroll

SCROLLING

API	EFFECT
`scrollIntoViewIfNeeded()`	Bring element into view
`mouse.wheel()`	Wheel scroll
`click({ scroll })`	Scroll behaviour option (1.62)

⚠ COMMON MISTAKES

- Using `page.keyboard.press` without focusing the right element first; keys go to the wrong place
- Hard-coding pixel coordinates for normal UI; they break on viewport or layout changes
- Expecting `dragTo` to work with libraries that need many intermediate moves; use `mouse` with `steps`

✓ TIPS & BEST PRACTICES

- Prefer `locator.press()` over `page.keyboard.press()` so focus is explicit
- Use `ControlOrMeta` for cross-platform shortcuts
- For canvas apps, compute coordinates from `locator.boundingBox()`

QUICK SUMMARY

- `press('Key+Chord')` — keyboard shortcuts
- `keyboard.down/up` — held keys
- `mouse.move/down/up` — manual drag
- `dragTo()` — one-line drag-and-drop

Sheet 17

Dialogs, Downloads & New Pages

page.on('dialog') · accept() · waitForEvent('download') · saveAs() · popups · context.waitForEvent('page')

EVENTS: DIALOGS, DOWNLOADS, POPUPS

Native `alert`/`confirm`/`prompt` dialogs are **auto-dismissed** unless you register a `dialog` handler. Downloads, popups and new tabs are events: start waiting with `page.waitForEvent()` (or `context.waitForEvent('page')`) **before** the action that triggers them, then await the promise. Downloads are saved to a temp folder and deleted when the context closes unless you `saveAs()`.

NOTES

- `page.once('dialog', …)` handles exactly one dialog
- Dialog handlers must accept or dismiss, or the page hangs
- `beforeunload` dialogs need `page.close({ runBeforeUnload: true })`
- `acceptDownloads` is `true` by default in the test runner

EXAMPLES

```
// confirm() / prompt()
page.once('dialog', async (dialog) => {
  expect(dialog.message()).toContain('Delete item?');
  await dialog.accept();                  // dialog.accept('text') for prompt()
});
await page.getByRole('button', { name: 'Delete' }).click();

// download
const downloadPromise = page.waitForEvent('download');
await page.getByRole('link', { name: 'Export CSV' }).click();
const download = await downloadPromise;
expect(download.suggestedFilename()).toBe('report.csv');
await download.saveAs(test.info().outputPath(download.suggestedFilename()));

// popup / new tab
const popupPromise = page.waitForEvent('popup');
await page.getByRole('link', { name: 'Open docs' }).click();
const popup = await popupPromise;
await expect(popup).toHaveURL(/docs/);
```

CODE EXPLAINED

- → `once('dialog')` — register before the trigger
- → `accept()` / `dismiss()` — must be called
- → `waitForEvent('download')` — promise first, then click
- → `saveAs()` — persist before context closes
- → `'popup'` — window.open / target=_blank

DIALOG API

METHOD	RETURNS
`type()`	alert | confirm | prompt | beforeunload
`message()`	Dialog text
`defaultValue()`	prompt default
`accept(text?)`	Confirm / answer
`dismiss()`	Cancel

DOWNLOAD API

METHOD	RETURNS
`suggestedFilename()`	Server filename
`path()`	Temp file path
`saveAs(path)`	Copy to path
`createReadStream()`	Readable stream
`failure()`	Error or null

OTHER PAGE EVENTS

EVENT	FIRES WHEN
`page.on('popup')`	Page opens a window
`context.on('page')`	Any new page in context
`page.on('close')`	Page closes
`page.on('crash')`	Renderer crashes

⚠ COMMON MISTAKES

- `await page.waitForEvent('download')` **before** clicking; it waits forever because nothing triggers it
- Registering `page.on('dialog')` and never calling accept/dismiss; the page freezes
- Reading `download.path()` after the context closed; the temp file is gone

✓ TIPS & BEST PRACTICES

- `Promise.all([waitForEvent, click])` is a compact alternative
- Use `context.waitForEvent('page')` when you don't know which page opens the tab
- Assert dialog text inside the handler with `expect`

QUICK SUMMARY

- `once('dialog')` — handle alerts/confirms
- `waitForEvent('download')` — capture files
- `waitForEvent('popup')` — capture new tabs
- `saveAs()` — keep downloaded files

Web-First Locator Assertions

toBeVisible · toHaveText · toContainText · toHaveValue · toBeEnabled · toHaveCount · toHaveAttribute · toHaveCSS

AUTO-RETRYING ASSERTIONS

Web-first assertions take a locator and **retry** until the condition holds or the expect timeout (5 s default) runs out. They are async, so they must be awaited. This makes them the right tool for UI state: never read a value with `textContent()` and compare it with a plain `expect`, which checks once and races the page.

NOTES

- Negate with `.not` : `expect(l).not.toBeVisible()`
- Per-call timeout: `toBeVisible({ timeout: 10_000 })`
- `toHaveText` = full match; `toContainText` = substring
- Arrays check lists in order: `toHaveText(['a', 'b'])`

EXAMPLES

```
const toast = page.getByRole('status');
await expect(toast).toBeVisible();
await expect(toast).toHaveText('Saved');                // exact (normalized)
await expect(toast).toContainText('Sav');
await expect(page.getByRole('listitem')).toHaveCount(3);
await expect(page.getByRole('listitem')).toHaveText(['One', 'Two', 'Three']);
await expect(page.getByLabel('Email')).toHaveValue('ada@example.com');
await expect(page.getByRole('button', { name: 'Pay' })).toBeEnabled();
await expect(page.getByRole('link')).toHaveAttribute('href', '/docs');
await expect(page.getByTestId('card')).toHaveClass(/selected/);
await expect(page.getByTestId('banner')).toHaveCSS('color', 'rgb(255, 0, 0)');
await expect(page.getByTestId('ok')).toHaveCSS('content', '"✓"',
  { pseudo: 'before' });
await expect(page.getByRole('dialog')).not.toBeVisible();   // or toBeHidden()
await expect(page.getByRole('button')).toHaveAccessibleName('Close');
```

CODE EXPLAINED

- → `await expect(locator)` — retries until pass/timeout
- → text matchers — string, RegExp or array
- → `toHaveAttribute` — name + optional value
- → `pseudo` — check `::before/::after` (1.60)
- → `.not` — waits for the opposite state

STATE MATCHERS

MATCHER	PASSES WHEN
`toBeVisible / toBeHidden`	Rendered / not
`toBeEnabled / toBeDisabled`	Interactivity
`toBeChecked`	Checkbox/radio checked
`toBeEditable`	Not readonly
`toBeFocused`	Has focus
`toBeEmpty`	No text / empty input
`toBeAttached`	In DOM (maybe hidden)
`toBeInViewport`	Intersects viewport

CONTENT MATCHERS

MATCHER	CHECKS
`toHaveText / toContainText`	Text content
`toHaveValue / toHaveValues`	Input / select values
`toHaveCount`	Number of matches
`toHaveAttribute`	Attribute value
`toHaveClass / toContainClass`	class attribute
`toHaveCSS`	Computed style
`toHaveId / toHaveRole`	id / ARIA role
`toHaveAccessibleName`	Computed name

⚠ COMMON MISTAKES

- `expect(await l.textContent()).toBe('Saved')` ; reads once and races, use `await expect(l).toHaveText('Saved')`
- Forgetting `await` before `expect(locator)` ; the assertion never runs
- `toHaveText('Sav')` expecting a substring; it's a full match, use `toContainText`

✓ TIPS & BEST PRACTICES

- Assert the outcome users see (toast, URL, heading), not internal state
- Raise the timeout on the one slow assertion, not globally
- Use array forms to check list order in one assertion

QUICK SUMMARY

- `await expect(locator).toX()` — auto-retrying
- `toHaveText / toContainText` — full / partial text
- `toHaveCount` — list size
- `.not` — negate with retry

Page & Generic Assertions

PAGE-LEVEL AND PLAIN VALUE ASSERTIONS

Page assertions (`toHaveURL`, `toHaveTitle`, `toHaveScreenshot`) retry like locator assertions. **Generic** matchers (`toBe`, `toEqual`, `toMatchObject`, `toContain`, …) work on plain values, **check once** and are synchronous — use them for data you already have, such as API responses or computed values. API responses also get `toBeOK()`.

NOTES

- Generic matchers follow Jest semantics
- `expect.configure({ timeout, soft })` creates a preconfigured `expect`
- Global expect timeout: `expect: { timeout: 10_000 }` in config
- Custom failure message: `expect(value, 'message')`

EXAMPLES

```ts
await expect(page).toHaveURL('/dashboard');           // relative to baseURL
await expect(page).toHaveURL(/\/orders\/\d+$/);
await expect(page).toHaveURL((u) => u.searchParams.get('tab') === 'billing');
await expect(page).toHaveTitle(/Checkout/);

const res = await page.request.get('/api/user');
await expect(res).toBeOK();                                  // 2xx
const user = await res.json();
expect(user).toEqual({ id: 7, name: 'Ada', roles: ['admin'] });   // deep equal
expect(user).toMatchObject({ name: 'Ada' });              // subset
expect(user.roles).toContain('admin');
expect(user.id).toBeGreaterThan(0);
expect(user).toHaveProperty('name', 'Ada');
expect(user.createdAt).toEqual(expect.any(String));

const slowExpect = expect.configure({ timeout: 15_000 });
await slowExpect(page.getByText('Report ready')).toBeVisible();
expect(user.name, 'user should be renamed after PATCH').toBe('Ada L.');
```

CODE EXPLAINED

- → `toHaveURL` — string, RegExp or predicate
- → `toBeOK()` — APIResponse status 200–299
- → `toEqual` — deep equality
- → `toMatchObject` — partial match
- → `expect.any()` — asymmetric matcher
- → `expect.configure()` — reusable timeout/soft

PAGE MATCHERS (RETRY)

MATCHER	CHECKS
`toHaveURL`	Current URL
`toHaveTitle`	document.title
`toHaveScreenshot`	Visual comparison
`toMatchAriaSnapshot`	Accessibility tree (1.60)

GENERIC MATCHERS (ONCE)

MATCHER	CHECKS
`toBe / toEqual`	Identity / deep equality
`toStrictEqual`	Deep + types/undefined
`toContain / toContainEqual`	Array / string item
`toHaveLength`	.length
`toBeTruthy / toBeNull`	Truthiness / null
`toThrow`	Function throws
`toBeCloseTo`	Float equality

⚠ COMMON MISTAKES

- `expect(page.url()).toBe('/x')` right after a click; it doesn't wait, use `await expect(page).toHaveURL('/x')`
- Using `toBe` on objects; reference equality fails, use `toEqual`
- Awaiting generic matchers or not awaiting page matchers; only the retrying ones are async

✓ TIPS & BEST PRACTICES

- Rule: locator or page → `await expect(...)`; plain value → `expect(...)`
- Use `toMatchObject` on API responses so new fields don't break tests
- Name important assertions with the message argument for clearer reports

QUICK SUMMARY

- `toHaveURL / toHaveTitle` — retrying page checks
- `toEqual / toMatchObject` — data checks
- `toBeOK()` — API status
- `expect.configure()` — custom timeout/soft defaults

Soft, Polling & Custom Assertions

expect.soft · expect.poll · expect.soft.poll · toPass() · expect.extend · custom message

BEYOND BASIC ASSERTIONS

Soft assertions record a failure but let the test continue, so one run reports several problems. `expect.poll()` turns any async function into a retrying assertion, and `expect(fn).toPass()` retries a whole block until it stops throwing. `expect.extend()` adds custom matchers that show up in reports like built-ins.

NOTES

- Test still fails at the end if any soft assertion failed
- `expect.soft.poll()` combines both (1.61)
- `toPass` default timeout is 0 (unbounded until test timeout); set one
- Check `test.info().errors.length` to stop early after soft failures

EXAMPLES

```ts
await expect.soft(page.getByTestId('status')).toHaveText('Success');
await expect.soft(page.getByTestId('eta')).toHaveText('2 days');
expect(test.info().errors).toHaveLength(0);              // optional: stop here

await expect.poll(async () => {
  const res = await page.request.get('/api/jobs/42');
  return (await res.json()).state;
}, { message: 'job should finish', timeout: 30_000, intervals: [1_000, 2_000] })
  .toBe('done');

await expect(async () => {
  const res = await page.request.get('/api/health');
  expect(res.status()).toBe(200);
}).toPass({ timeout: 20_000, intervals: [1_000] });

// custom matcher
expect.extend({
  async toHaveAmount(locator: Locator, expected: number) {
    const text = await locator.textContent();
    const pass = Number(text?.replace(/[^\d.]/g, '')) === expected;
    return { pass, message: () => `expected amount ${expected}, got ${text}` };
  },
});
```

CODE EXPLAINED

- → `expect.soft` — keep going after failure
- → `expect.poll(fn)` — retry a value-producing function
- → `intervals` — polling back-off in ms
- → `toPass()` — retry a block of assertions
- → `expect.extend` — add `toHaveAmount` matcher

WHICH TOOL?

NEED	USE
`Wait for UI state`	await expect(locator)
`Wait for backend value`	expect.poll()
`Retry several steps`	expect(fn).toPass()
`Report all mismatches`	expect.soft()
`Domain-specific check`	expect.extend()

POLL / TOPASS OPTIONS

OPTION	MEANING
`timeout`	Give up after ms
`intervals`	Retry delays, last repeats
`message`	Shown on failure

⚠ COMMON MISTAKES

- Using `toPass()` without a timeout; a never-passing block runs until the test timeout
- Wrapping `page.click()` inside `toPass` to "make it pass"; fix the locator or wait instead
- Thinking soft failures make a test pass; they still fail it at the end

✓ TIPS & BEST PRACTICES

- Use soft assertions for page-content audits with many independent checks
- Prefer `expect.poll` over `waitForTimeout` loops for eventual consistency
- Export custom matchers from your fixtures file with `mergeExpects()`

QUICK SUMMARY

- `expect.soft()` — non-fatal checks
- `expect.poll()` — retry any async value
- `toPass()` — retry assertion blocks
- `expect.extend()` — custom matchers

Navigation & Load States

goto() · waitUntil · baseURL · waitForURL() · waitForLoadState() · reload() · goBack()

NAVIGATING PAGES

`page.goto()` loads a URL and waits for the `load` event by default. With `baseURL` set in config, pass paths like `'/login'`. After a click causes navigation, you rarely need explicit waits: the next locator action or `toHaveURL` assertion waits for the new page. Use `waitForURL()` when you need to block until a redirect chain finishes.

NOTES

- `goto` returns the main `Response` (or `null` for same-document)
- `goto` throws on network errors, not on 404/500 status
- `networkidle` is discouraged; assert on content instead
- SPAs: route changes are not loads; assert URL or content

EXAMPLES

```
await page.goto('/login');                                  // baseURL + /login
const res = await page.goto('/products/42');
expect(res?.status()).toBe(200);
await page.goto('/', { waitUntil: 'domcontentloaded' });  // don't wait for images
await page.goto('https://other.site', { referer: 'https://google.com' });

await page.getByRole('button', { name: 'Sign in' }).click();
await page.waitForURL('**/dashboard');                      // glob
await page.waitForURL(/\/orders\/\d+/, { waitUntil: 'commit' });
await expect(page).toHaveURL('/dashboard');                // usually enough

await page.waitForLoadState('domcontentloaded');            // after popup/new page
await page.reload();
await page.goBack();
await page.goForward();
await page.setContent('<button>Hi</button>');               // inline HTML
```

CODE EXPLAINED

- → `goto(path)` — resolved against `baseURL`
- → `waitUntil` — which event counts as loaded
- → `waitForURL` — glob, RegExp or predicate
- → `toHaveURL` — assertion that also waits
- → `setContent` — render an HTML string

WAITUNTIL VALUES

VALUE	WAITS FOR
`load`	window load event (default)
`domcontentloaded`	DOMContentLoaded
`networkidle`	500 ms no network (avoid)
`commit`	Response received, doc starts

NAVIGATION APIS

API	PURPOSE
`page.goto(url, opts)`	Navigate
`page.waitForURL(url)`	Wait for URL match
`page.waitForLoadState(s)`	Wait for state
`page.reload() / goBack()`	History
`page.url()`	Current URL (no wait)

⚠ COMMON MISTAKES

- `waitForNavigation()` in new code; it's deprecated and racy, use `waitForURL` or assertions
- `waitUntil: 'networkidle'` on apps with polling or websockets; it never settles
- Checking `page.url()` right after a click; it may still be the old URL

✓ TIPS & BEST PRACTICES

- Set `baseURL` so tests run against any environment with one config change
- Use `domcontentloaded` to speed up pages with heavy images or ads
- After login redirects, assert a heading on the target page instead of the URL alone

QUICK SUMMARY

- `goto(path)` — navigate with baseURL
- `waitUntil` — load, domcontentloaded, commit
- `waitForURL()` — block for redirects
- `toHaveURL()` — preferred URL check

Auto-Waiting & Timeouts

actionability checks · test timeout · expect timeout · actionTimeout · navigationTimeout · signal (AbortSignal) · timeout: 0

HOW WAITING WORKS

Playwright waits in three places: **actions** wait for actionability, **web-first assertions** retry until they pass, and **navigation** waits for a load state. Each wait is bounded by a timeout. Since 1.62 you can also cancel actions and assertions with an `AbortSignal`, and `timeout: 0` disables the default for that call. Hard sleeps (`waitForTimeout`) are almost never needed.

DEFAULT TIMEOUTS

- Test: **30 s** (includes hooks and fixtures)
- Expect: **5 s** per assertion
- Action / navigation: **no limit** (bounded by test timeout)
- Global run: unlimited (`globalTimeout`)

CONFIGURING TIMEOUTS

```ts
// playwright.config.ts
export default defineConfig({
  timeout: 60_000,                        // per test
  globalTimeout: 30 * 60_000,             // whole run (CI safety net)
  expect: { timeout: 10_000 },            // per assertion
  use: { actionTimeout: 15_000, navigationTimeout: 30_000 },
});

// per test / per call
test('big export', async ({ page }) => {
  test.setTimeout(120_000);               // or test.slow() → 3×
  await page.getByRole('button', { name: 'Export' }).click({ timeout: 5_000 });
  await expect(page.getByText('Ready')).toBeVisible({ timeout: 60_000 });

  const ctrl = new AbortController();
  setTimeout(() => ctrl.abort(), 2_000);
  await page.getByText('Maybe').click({ signal: ctrl.signal, timeout: 0 });  // 1.62
});
```

CODE EXPLAINED

- → `timeout` — whole test incl. hooks
- → `expect.timeout` — each assertion
- → `actionTimeout` — each click/fill
- → `setTimeout()` / `slow()` — per-test override
- → `signal` — cancel externally (1.62)

TIMEOUT REFERENCE

SETTING	DEFAULT
`timeout`	30 000 ms
`expect.timeout`	5 000 ms
`actionTimeout`	0 (none)
`navigationTimeout`	0 (none)
`globalTimeout`	0 (none)
`webServer.timeout`	60 000 ms

EXPLICIT WAITS (RARELY NEEDED)

API	USE
`locator.waitFor({ state })`	attached / visible / hidden
`page.waitForResponse()`	Network response
`page.waitForFunction()`	JS predicate in page
`locator.waitForFunction()`	Predicate on element (1.62)
`page.waitForTimeout(ms)`	Debugging only

⚠ COMMON MISTAKES

- Raising the global `timeout` to hide one slow page; fix or `test.slow()` that test
- `waitForTimeout(3000)` sleeps; they slow every run and still flake
- Setting `actionTimeout` higher than the test timeout; the test times out first

✓ TIPS & BEST PRACTICES

- Set `globalTimeout` in CI so a hung run fails instead of burning minutes
- Read "Timeout 5000ms exceeded" errors: they say which assertion and what was received
- Prefer assertions over `waitFor()` ; they explain failures better

QUICK SUMMARY

- `auto-wait` — actions + assertions wait
- `timeout / expect.timeout` — test vs assertion
- `test.slow()` — 3× for one test
- `signal` — cancel any action (1.62)

Built-in Fixtures

page · context · browser · browserName · request · mount · testInfo

WHAT FIXTURES ARE

Fixtures are the values a test destructures from its first argument. Playwright creates only the fixtures a test asks for, sets them up before the test and tears them down after, even on failure. Test-scoped fixtures (`page`, `context`) are fresh per test; worker-scoped ones (`browser`) are shared by all tests in one worker process.

NOTES

- Fixtures are lazy: unused fixtures cost nothing
- `context` options come from `use` in config or `test.use()`
- `mount` is built in since 1.62 (component testing)
- `testInfo` is the second argument, not a fixture

USING BUILT-INS

```
test('two users chat', async ({ browser, browserName, request }) => {
  test.skip(browserName === 'webkit', 'WS issue on WebKit');

  const alice = await browser.newContext();          // extra isolated users
  const bob = await browser.newContext({ storageState: 'playwright/.auth/bob.json' });
  const a = await alice.newPage();
  const b = await bob.newPage();

  const res = await request.post('/api/rooms', { data: { name: 'qa' } });
  const { id } = await res.json();
  await a.goto(`/rooms/${id}`);
  await b.goto(`/rooms/${id}`);
  await a.getByRole('textbox').fill('hi bob');
  await a.keyboard.press('Enter');
  await expect(b.getByText('hi bob')).toBeVisible();
  await alice.close(); await bob.close();
});
```

CODE EXPLAINED

- → `browser` — create extra contexts per user
- → `browserName` — 'chromium' \| 'firefox' \| 'webkit'
- → `request` — API client sharing `baseURL`
- → manual contexts — you must close them

BUILT-IN FIXTURES

FIXTURE	SCOPE · PROVIDES
`page`	test · Page in a fresh context
`context`	test · BrowserContext for page
`browser`	worker · Shared Browser
`browserName`	worker · Engine name
`request`	test · APIRequestContext
`mount`	test · Component testing mount (1.62)
`playwright`	worker · Playwright API object

OPTION FIXTURES (SET VIA USE)

OPTION	EXAMPLE
`baseURL`	'http://localhost:3000'
`storageState`	'playwright/.auth/user.json'
`viewport`	{ width: 1280, height: 720 }
`locale / timezoneId`	'en-GB' / 'Europe/London'
`headless / channel`	false / 'chrome'
`trace / video / screenshot`	'on-first-retry'

⚠ COMMON MISTAKES

- Creating contexts with `browser.newContext()` and never closing them; memory leaks across the worker
- Asking for `page` in a pure API test; it launches a browser for nothing, use `request`
- Mutating `browser` state (e.g. closing it) inside a test

✓ TIPS & BEST PRACTICES

- Destructure only what you need; it keeps tests fast
- Use `browser.newContext()` for multi-user scenarios
- Use `request` for setup data instead of clicking through the UI

QUICK SUMMARY

- `page / context` — per-test isolation
- `browser` — shared per worker
- `request` — API calls in tests
- `use: {}` — configure option fixtures

Custom Fixtures

WRITING YOUR OWN FIXTURES

`test.extend()` returns a new `test` with extra fixtures. Each fixture is an async function that does setup, hands a value to the test with `await use(value)`, then runs teardown after `use` returns. Fixtures can depend on other fixtures, be **worker-scoped** for expensive resources, run **automatically**, or act as configurable **options**.

NOTES

- Export the extended `test` / `expect` from one `fixtures.ts`
- Code after `use()` is teardown and runs even on failure
- Worker fixtures can only depend on worker fixtures
- Override built-ins too: `page: async ({ page }, use) => …`

FIXTURES.TS

```ts
import { test as base, expect } from '@playwright/test';
import { TodoPage } from './pages/TodoPage';

type TestFx = { todoPage: TodoPage };
type WorkerFx = { dbUrl: string };

export const test = base.extend<TestFx, WorkerFx>({
  todoPage: async ({ page }, use) => {
    const todo = new TodoPage(page);
    await todo.goto();
    await use(todo);                                  // test runs here
    await todo.removeAll();                           // teardown
  },
  dbUrl: [async ({}, use, workerInfo) => {
    await use(`postgres://localhost/test_${workerInfo.workerIndex}`);
  }, { scope: 'worker' }],
});
export { expect };
```

CODE EXPLAINED

- → `base.extend<T, W>()` — test + worker fixture types
- → `await use(value)` — hand value to the test
- → after `use` — teardown
- → `[fn, { scope: 'worker' }]` — once per worker
- → `workerIndex` — unique per worker (data isolation)

FIXTURE OPTIONS

OPTION	EFFECT
`scope: 'worker'`	Shared by tests in a worker
`auto: true`	Runs even if not requested
`option: true`	Overridable via `use`
`timeout`	Separate fixture timeout
`box: true`	Hide from reports
`title`	Custom report title

COMPOSITION

API	USE
`mergeTests(a, b)`	Combine fixture sets
`mergeExpects(a, b)`	Combine custom matchers
`test.extend(...) chain`	Layer fixtures
`test.use({ x })`	Override per file

⚠ COMMON MISTAKES

- Forgetting `await use(...)` ; the test never receives the value and hangs
- Putting teardown before `use` ; it runs before the test
- Worker fixture depending on `page` ; worker scope can't use test-scoped fixtures

✓ TIPS & BEST PRACTICES

- Use `auto: true` fixtures for global hooks like console-error collection
- Make test data unique with `testInfo.workerIndex` or IDs to stay parallel-safe
- Prefer fixtures over `beforeEach` for anything with teardown

QUICK SUMMARY

- `test.extend()` — add fixtures
- `use(value)` — setup → test → teardown
- `scope: 'worker'` — share expensive resources
- `mergeTests()` — combine fixture modules

Page Object Model

class with Locator fields · constructor(page) · action methods · fixtures for POMs · component objects

PAGE OBJECTS

A **page object** wraps one page (or component) behind a small class: locators are fields created in the constructor, and user actions are methods. Tests then read like specs, and a markup change is fixed in one place. In Playwright, locators are lazy, so defining them up front costs nothing; expose page objects to tests through **fixtures**.

GUIDELINES

- Keep assertions in tests, not in page objects (mostly)
- Methods model user intent: `addItem()`, not `clickPlus()`
- Return other page objects on navigation for fluent flows
- Component objects (header, modal) can be shared by pages

LOGINPAGE + FIXTURE

```ts
// pages/LoginPage.ts
import { type Page, type Locator, expect } from '@playwright/test';
export class LoginPage {
  readonly email: Locator;
  readonly password: Locator;
  readonly submit: Locator;
  constructor(readonly page: Page) {
    this.email = page.getByLabel('Email');
    this.password = page.getByLabel('Password');
    this.submit = page.getByRole('button', { name: 'Sign in' });
  }
  async goto() { await this.page.goto('/login'); }
  async login(email: string, pw: string) {
    await this.email.fill(email);
    await this.password.fill(pw);
    await this.submit.click();
  }
}
// fixtures.ts
export const test = base.extend<{ loginPage: LoginPage }>({
  loginPage: async ({ page }, use) => { await use(new LoginPage(page)); },
});
```

CODE EXPLAINED

- → `readonly` locator fields — defined once, resolved lazily
- → `constructor(page)` — receives the test's page
- → `login()` — intent-level action
- → fixture — tests get `loginPage` ready-made

TEST USING IT

```ts
test('valid login', async ({ loginPage, page }) => {
  await loginPage.goto();
  await loginPage.login('ada@example.com', 's3cret');
  await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
});
```

WHY FIXTURES?

- → No `new LoginPage(page)` in every test
- → Fixtures can pre-navigate or seed data

⚠ COMMON MISTAKES

- Storing element handles (`$()`) instead of locators; handles go stale after re-render
- Giant "god" page objects for the whole app; split by page and component
- Hiding every assertion inside page objects; failures lose context

✓ TIPS & BEST PRACTICES

- Wrap page-object methods in `test.step()` for readable reports
- Use `getByRole` -based locators in page objects; they survive redesigns
- Generate first drafts with codegen, then refactor into page objects

QUICK SUMMARY

- `class XPage` — locators + actions
- `constructor(page)` — lazy locator setup
- fixture — inject page objects
- component objects — reuse shared UI

Global Setup & Project Dependencies

ONE-TIME SETUP

The recommended way to run something once before tests (seed a DB, log in) is a **setup project**: a normal project whose tests do the setup, listed in other projects' `dependencies`. Setup tests show in reports, get traces, and can use fixtures. The older `globalSetup` file still works for plain scripts. **Test locks** (1.63) serialize tests that share an external resource.

NOTES

- Dependencies run first; if they fail, dependents are skipped
- `teardown` names a project that runs after all dependents finish
- `--no-deps` skips dependencies when iterating locally
- `globalSetup` gets no fixtures, traces or report entry

PLAYWRIGHT.CONFIG.TS

```ts
export default defineConfig({
  projects: [
    { name: 'setup db', testMatch: /global\.setup\.ts/, teardown: 'cleanup db'},
    { name: 'cleanup db', testMatch: /global\.teardown\.ts/ },
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
      dependencies: ['setup db'],
    },
  ],
});

// tests/global.setup.ts
import { test as setup } from '@playwright/test';
setup('seed database', async ({ request }) => {
  await request.post('/api/test/reset');
});

// test locks: never run these two at the same time (1.63)
test('edits shared config', { lock: 'settings' }, async ({ page }) => {});
test.describe.configure({ lock: 'settings' });             // whole file (1.64)
```

CODE EXPLAINED

- → setup project — matched by `testMatch`
- → `dependencies` — run before this project
- → `teardown` — run after dependents finish
- → `{ lock }` — serialize tests sharing a resource

SETUP PROJECT VS GLOBALSETUP

FEATURE	SETUP PROJECT · GLOBALSETUP
`Fixtures (page, request)`	Yes · No
`HTML report + trace`	Yes · No
`Config use applied`	Yes · Manual
`Per-project setup`	Yes · No (global)
`Runs with --no-deps`	Skipped · Always

RELATED FLAGS

FLAG	EFFECT
`--no-deps`	Skip dependency projects
`--project=setup`	Run setup alone
`--project=chromium`	Pulls in its dependencies

⚠ COMMON MISTAKES

- Using `beforeAll` for once-per-run work; it runs once **per worker**
- Forgetting to add the setup file to `testMatch`; the default pattern doesn't match `*.setup.ts`
- Teardown that depends on state from setup variables; projects run in separate processes

✓ TIPS & BEST PRACTICES

- Put auth login in a setup project and save `storageState` (next sheet)
- Use `--no-deps` locally once setup data exists
- Use locks instead of `workers: 1` when only a few tests conflict

QUICK SUMMARY

- setup project + `dependencies` — run first
- `teardown` — run last
- `globalSetup` — legacy script hook
- `{ lock: 'name' }` — mutual exclusion

Auth with storageState

LOG IN ONCE, REUSE EVERYWHERE

Logging in through the UI in every test is slow. Instead, a **setup project** signs in once and saves cookies and local storage with `context.storageState({ path })`. Test projects load that file through `use.storageState`, so every test starts already authenticated. Since 1.59, `context.setStorageState()` can swap state into an existing context.

NOTES

- State file = cookies + localStorage (+ IndexedDB if asked)
- sessionStorage is **not** saved; see next sheet
- Add `playwright/.auth` to `.gitignore`
- Expired sessions: delete the file or re-run setup

TESTS/AUTH.SETUP.TS

```ts
import { test as setup, expect } from '@playwright/test';
const authFile = 'playwright/.auth/user.json';

setup('authenticate', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill(process.env.E2E_USER!);
  await page.getByLabel('Password').fill(process.env.E2E_PASS!);
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.getByRole('button', { name: 'Account' })).toBeVisible();
  await page.context().storageState({ path: authFile, indexedDB: true });
});
```

CODE EXPLAINED

- → wait for a logged-in signal before saving
- → `storageState({ path })` — write the file
- → `indexedDB: true` — for Firebase-style auth
- → credentials from env, never hard-coded

PLAYWRIGHT.CONFIG.TS

```ts
projects: [
  { name: 'setup', testMatch: /.*\.setup\.ts/ },
  {
    name: 'chromium',
    use: {
      ...devices['Desktop Chrome'],
      storageState: 'playwright/.auth/user.json',
    },
    dependencies: ['setup'],
  },
],
// opt a file out: test.use({ storageState: { cookies: [], origins: [] } });
```

CODE EXPLAINED

- → `storageState` in `use` — every test starts logged in
- → `dependencies` — setup runs first
- → empty state — test logged-out flows

⚠ COMMON MISTAKES

- Saving state before the login redirect finishes; the file has no session cookie
- Committing `.auth/*.json`; it's a live session token
- Sharing one account across parallel tests that mutate server state; they collide

✓ TIPS & BEST PRACTICES

- Log in via API (`request.post('/api/login')`) in setup for speed
- Use `test.use({ storageState: … })` per file for different roles
- Regenerate state in CI on every run; it's fast and avoids expiry

QUICK SUMMARY

- setup project — log in once
- `storageState({ path })` — save session
- `use.storageState` — load for tests
- empty state — logged-out tests

Multi-Role & Advanced Auth

multiple roles · per-worker accounts · API login · sessionStorage · passkeys (context.credentials) · httpCredentials

BEYOND ONE USER

Real apps need **several roles**, accounts that don't collide in parallel, and auth schemes beyond cookies. Save one state file per role and pick it per file or per context. For tests that mutate shared account data, use a **worker-scoped** fixture that logs in a distinct account per worker. Passkeys use the virtual WebAuthn authenticator `context.credentials` (1.61).

NOTES

- HTTP Basic auth: `use: { httpCredentials: { username, password } }`
- sessionStorage: read with `page.evaluate`, restore via `addInitScript`
- Or use `page.sessionStorage` / `page.localStorage` APIs (1.61)
- Client certificates: `use: { clientCertificates: [...] }`

PATTERNS

```
// 1. two roles in one test
test('admin approves user request', async ({ browser }) => {
  const admin = await browser.newContext({ storageState: '.auth/admin.json' });
  const user = await browser.newContext({ storageState: '.auth/user.json' });
  /* … drive both pages, then close both contexts … */
});

// 2. one account per parallel worker
export const test = base.extend<{}, { workerStorageState: string }>({
  storageState: ({ workerStorageState }, use) => use(workerStorageState),
  workerStorageState: [async ({ browser }, use, { workerIndex }) => {
    const file = `playwright/.auth/worker-${workerIndex}.json`;
    const ctx = await browser.newContext({ storageState: undefined });
    const res = await ctx.request.post('/api/login',
      { data: { user: `qa${workerIndex}`, pass: process.env.E2E_PASS } });
    await ctx.storageState({ path: file }); await ctx.close();
    await use(file);
  }, { scope: 'worker' }],
});
```

CODE EXPLAINED

- → per-role state files — switch identity per context
- → override `storageState` fixture — inject worker file
- → `ctx.request` — API login shares cookies with ctx
- → `workerIndex` — unique account per worker

AUTH SCHEMES

SCHEME	APPROACH
`Cookie session`	storageState file
`Token in localStorage`	storageState file
`Token in sessionStorage`	addInitScript restore
`HTTP Basic`	use.httpCredentials
`Bearer API token`	extraHTTPHeaders
`Passkeys / WebAuthn`	context.credentials (1.61)
`mTLS`	use.clientCertificates

PER-ROLE SELECTION

WHERE	HOW
`Whole project`	use.storageState
`One file`	test.use({ storageState })
`One context`	browser.newContext({ storageState })

⚠ COMMON MISTAKES

- Many parallel tests changing the same user's settings; use per-worker accounts
- Logging in through the UI inside the worker fixture; API login is faster and less flaky
- Assuming `storageState` captures sessionStorage; it doesn't

✓ TIPS & BEST PRACTICES

- Keep role files named by role (`admin.json`, `viewer.json`)
- Use `ctx.request` so API login cookies land in the browser context
- Seed test accounts in the setup project before workers start

QUICK SUMMARY

- one state file per role
- worker fixture — account per worker
- `httpCredentials` — Basic auth
- `context.credentials` — passkeys

Routing, Mocking & HAR

page.route() · route.fulfill() · route.continue() · route.abort() · route.fetch() · routeFromHAR() · tracing.startHar()

INTERCEPTING NETWORK TRAFFIC

`page.route()` (or `context.route()`) intercepts requests matching a glob, RegExp or predicate. The handler can **fulfill** with a fake response, **continue** with modified headers, **abort**, or **fetch** the real response and patch it. For whole flows, record traffic to a **HAR** file once and replay it with `routeFromHAR()` so tests run without the backend.

NOTES

- Register routes **before** the navigation that triggers them
- Last registered matching route wins; `route.fallback()` passes on
- Service workers can bypass routing; set `serviceWorkers: 'block'`
- `page.unroute()` / `unrouteAll()` remove handlers

EXAMPLES

```
// mock a JSON API
await page.route('**/api/products', (route) =>
  route.fulfill({ status: 200, json: [{ id: 1, name: 'Mug', price: 9 }] }));

// patch a real response
await page.route('**/api/user', async (route) => {
  const response = await route.fetch();
  const json = await response.json();
  await route.fulfill({ response, json: { ...json, plan: 'pro' } });
});

// block images, force an error
await page.route(/\.(png|jpe?g|webp)$/, (route) => route.abort());
await page.route('**/api/checkout', (route) => route.fulfill({ status: 500 }));

// HAR: record with update: true, then replay
await page.routeFromHAR('hars/shop.har', { url: '**/api/**', update: false });
```

CODE EXPLAINED

- → `fulfill({ json })` — fake response body
- → `route.fetch()` — get real response to patch
- → `abort()` — simulate network failure
- → `routeFromHAR` — replay recorded traffic

ROUTE METHODS

METHOD	EFFECT
`fulfill(opts)`	Respond without network (`status`, `json`, `body`, `path`)
`continue(opts)`	Send request, optionally changing `url`, `headers`, `postData`
`fetch()`	Perform it, return the response
`abort(code?)`	Fail the request
`fallback()`	Defer to next handler

HAR OPTIONS

OPTION / API	PURPOSE
`update: true`	Record/refresh the HAR file
`url`	Only serve matching URLs
`notFound: 'fallback'`	Unmatched → real network
`tracing.startHar()`	Record HAR via tracing (1.60)

⚠ COMMON MISTAKES

- Adding the route after `goto`; the request already went out
- Glob `'/api/products'` without `**/`; it must match the full URL
- Forgetting to re-record HARs after API changes; replays drift from reality

✓ TIPS & BEST PRACTICES

- Mock only third-party or flaky dependencies; keep your own API real in e2e
- Use `route.fetch()` + patch to test edge cases with real data shapes
- Use `context.route()` so popups and new pages are mocked too

QUICK SUMMARY

- `route(url, handler)` — intercept
- `fulfill / continue / abort` — respond, pass, fail
- `route.fetch()` — patch real responses
- `routeFromHAR()` — replay recorded traffic

Waiting on Network Traffic

OBSERVING REQUESTS

Sometimes a test needs to wait for or inspect a specific call: confirm the form actually POSTed, grab an ID from a response, or log failures. Start `waitForResponse()` **before** the triggering action. Event listeners (`page.on('request' | 'response' | 'requestfailed')`) observe everything, and `routeWebSocket()` mocks or inspects WebSocket traffic.

NOTES

- Match by URL glob, RegExp or predicate on the Response
- `response.ok()` is true for 200–299
- HAR and trace recordings include WebSockets (1.61)
- `page.requests()` / `consoleMessages()` read buffered history

EXAMPLES

```
const saved = page.waitForResponse((r) =>
  r.url().endsWith('/api/orders') && r.request().method() === 'POST');
await page.getByRole('button', { name: 'Place order' }).click();
const res = await saved;
expect(res.status()).toBe(201);
const { id } = await res.json();

const reqPromise = page.waitForRequest('**/api/search?q=*');
await page.getByRole('searchbox').fill('mug');
expect((await reqPromise).url()).toContain('q=mug');

page.on('requestfailed', (r) => console.log('FAILED', r.url()));
page.on('response', (r) => r.status() >= 500 && console.log('5xx', r.url()));

await page.routeWebSocket('wss://example.com/live', (ws) => {
  ws.onMessage((msg) => msg === 'ping' && ws.send('pong'));   // fake server
});
```

CODE EXPLAINED

- → `waitForResponse(predicate)` — promise before click
- → `res.json()` — parsed body
- → `waitForRequest(glob)` — inspect outgoing call
- → `on('requestfailed')` — log network errors
- → `routeWebSocket` — mock a WS server

REQUEST API

METHOD	RETURNS
`url()` / `method()`	URL / verb
`headers()`	Headers
`postDataJSON()`	Parsed body
`resourceType()`	xhr, fetch…
`response()`	Response | null

RESPONSE API

METHOD	RETURNS
`status()` / `ok()`	Code / 2xx flag
`json()` / `text()` / `body()`	Body
`headers()`	Header map
`request()`	Originating request

PAGE EVENTS

EVENT	FIRES ON
`request`	Request issued
`response`	Response received
`requestfinished`	Body downloaded
`requestfailed`	Network error
`websocket`	WS opened

⚠ COMMON MISTAKES

- `await page.waitForResponse(...)` before clicking; it blocks forever
- Matching only the URL when GET and POST share it; check `request().method()` too
- Asserting on a response that the UI hasn't rendered yet; also assert the UI

✓ TIPS & BEST PRACTICES

- Use response waits to grab server-generated IDs for later steps
- Add an `auto` fixture that fails tests on unexpected 5xx responses
- Prefer UI assertions; network waits are for data you can't see

QUICK SUMMARY

- `waitForResponse()` — wait + inspect
- `waitForRequest()` — verify outgoing calls
- `page.on('…')` — observe all traffic
- `routeWebSocket()` — mock WebSockets

API Testing

request fixture · request.get/post · apiRequestContext · extraHTTPHeaders · storageState sharing · cookie methods

TESTING APIS DIRECTLY

The `request` fixture is an **APIRequestContext**: an HTTP client that uses `baseURL`, `extraHTTPHeaders` and proxy settings from config. Use it for pure API tests, or inside UI tests to seed data and verify the backend. `page.request` shares cookies with the browser context, so API calls are authenticated like the user.

NOTES

- `request` fixture has its own cookie jar; `page.request` shares the page's
- `data` objects are sent as JSON; use `form` or `multipart` otherwise
- `failOnStatusCode: true` throws on non-2xx
- `response.timing()` (1.62) and cookie methods (1.64) available

API TESTS

```
// playwright.config.ts → use: { baseURL: 'https://api.example.com',
//   extraHTTPHeaders: { Authorization: `Bearer ${process.env.API_TOKEN}` } }

test('create and fetch an issue', async ({ request }) => {
  const created = await request.post('/repos/acme/app/issues', {
    data: { title: 'Bug: cart total', labels: ['bug'] },
  });
  expect(created.status()).toBe(201);
  const { number } = await created.json();

  const got = await request.get(`/repos/acme/app/issues/${number}`);
  await expect(got).toBeOK();
  expect(await got.json()).toMatchObject({ state: 'open' });
});

test('UI shows API-created item', async ({ page }) => {
  await page.request.post('/api/todos', { data: { text: 'from API' } });
  await page.goto('/todos');
  await expect(page.getByText('from API')).toBeVisible();
});
```

CODE EXPLAINED

- → `request.post(url, { data })` — JSON body
- → `toBeOK()` — 2xx check with good errors
- → `toMatchObject` — partial body match
- → `page.request` — authenticated as the page

REQUEST METHODS

METHOD	OPTIONS
`get / delete / head`	params, headers, timeout
`post / put / patch`	data, form, multipart
`fetch(url, { method })`	Any verb
`dispose()`	Close a manual context
`storageState()`	Save cookies for UI tests

STANDALONE CONTEXT

CODE	PURPOSE
`request.newContext({ baseURL })`	Separate client
`{ extraHTTPHeaders }`	Default headers
`{ storageState }`	Reuse UI session
`{ ignoreHTTPSErrors }`	Self-signed certs

⚠ COMMON MISTAKES

- Using the `request` fixture expecting the UI's login cookies; use `page.request`
- Forgetting `await` on `res.json()`; you compare a Promise
- Leaking contexts from `request.newContext()`; call `dispose()`

✓ TIPS & BEST PRACTICES

- Seed data with API calls and verify through the UI for faster e2e tests
- Keep API tests in a separate project with no browser (`request` only)
- Log in via API, then `storageState()` for UI tests

QUICK SUMMARY

- `request` fixture — HTTP client
- `page.request` — shares browser cookies
- `toBeOK()` — status assertion
- `newContext()` — custom API clients

Browsers & Channels

CHOOSING BROWSERS

Cross-browser runs are configured as **projects**, one per browser or device. By default Playwright uses its own Chromium build; set `channel` to run branded **Google Chrome** or **Microsoft Edge** that you install with `npx playwright install chrome`. Headless Chromium runs the lighter **chromium-headless-shell** unless you choose `channel: 'chromium'` for the full new headless mode.

NOTES

- `npx playwright test --project=firefox` runs one project
- `launchOptions.args` passes Chromium flags
- `slowMo` slows every action (debugging)
- `connectOverCDP()` attaches to an existing Chrome (Chromium only)

PLAYWRIGHT.CONFIG.TS

```ts
import { defineConfig, devices } from '@playwright/test';
export default defineConfig({
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox',  use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit',   use: { ...devices['Desktop Safari'] } },
    { name: 'chrome',   use: { ...devices['Desktop Chrome'], channel: 'chrome' } },
    { name: 'edge',     use: { ...devices['Desktop Edge'], channel: 'msedge' } },
    {
      name: 'chromium-debug',
      use: {
        headless: false,
        launchOptions: { slowMo: 250, args: ['--disable-gpu'] },
      },
    },
  ],
});
// attach to a running Chrome started with --remote-debugging-port=9222
// const browser = await chromium.connectOverCDP('http://localhost:9222');
```

CODE EXPLAINED

- → `devices[...]` — UA, viewport, scale for a browser
- → `channel` — branded browser instead of bundled
- → `headless: false` — show the window
- → `slowMo` — ms pause after each action
- → `connectOverCDP` — drive an existing browser

CHANNELS

CHANNEL	BROWSER
`(none)`	Bundled Chromium (headless shell when headless)
`chromium`	Bundled Chromium, new headless mode
`chrome / chrome-beta`	Google Chrome stable / beta
`msedge / msedge-beta`	Microsoft Edge stable / beta

LAUNCHOPTIONS

OPTION	EFFECT
`args`	Extra browser flags
`slowMo`	Delay per action (ms)
`executablePath`	Custom browser binary
`downloadsPath`	Where downloads go
`proxy`	Browser-level proxy

⚠ COMMON MISTAKES

- Testing only Chromium, then shipping Safari bugs; add a WebKit project
- Using `channel: 'chrome'` in CI without `npx playwright install chrome`
- Overriding `devices` fields in the wrong order; spread `...devices[x]` first, then overrides

✓ TIPS & BEST PRACTICES

- Run all browsers on CI and only Chromium locally for speed
- Use branded `chrome` / `msedge` when testing codecs, DRM or enterprise policies
- Add a headed `slowMo` project for demos and debugging

QUICK SUMMARY

- one project per browser
- `channel` — branded Chrome/Edge
- `launchOptions` — flags, slowMo
- `connectOverCDP` — attach to running Chrome

Devices & Emulation

EMULATING DEVICES AND ENVIRONMENTS

The `devices` registry bundles user agent, viewport, device scale factor, touch and `isMobile` for hundreds of phones, tablets and desktops. On top of that, any context can emulate **locale**, **time zone**, **geolocation**, **color scheme**, **reduced motion**, **forced colors** and **contrast**, either globally in `use`, per file with `test.use()`, or at runtime with `page.emulateMedia()`.

NOTES

- Mobile emulation runs in desktop engines (WebKit ≈ Safari iOS)
- `isMobile` is Chromium-only; Firefox ignores it
- Since 1.64, `screen` is copied from device descriptors
- Geolocation needs `permissions: ['geolocation']`

EXAMPLES

```ts
// playwright.config.ts
projects: [
  { name: 'Mobile Safari', use: { ...devices['iPhone 15'] } },
  { name: 'Pixel', use: { ...devices['Pixel 7'] } },
],

// per file
test.use({
  locale: 'fr-FR',
  timezoneId: 'Europe/Paris',
  geolocation: { latitude: 48.8584, longitude: 2.2945 },
  permissions: ['geolocation'],
  colorScheme: 'dark',
  reducedMotion: 'reduce',            // also contrast / forcedColors (1.63)
});

// at runtime
await page.emulateMedia({ colorScheme: 'light', media: 'print' });
await page.setViewportSize({ width: 375, height: 812 });
await context.setOffline(true);
```

CODE EXPLAINED

- → `devices[name]` — full device profile
- → `locale` / `timezoneId` — Intl + Date behaviour
- → `geolocation` + permission — fake GPS
- → `colorScheme` — prefers-color-scheme
- → `emulateMedia` — switch at runtime (print, dark)

EMULATION OPTIONS

OPTION	VALUES
`viewport`	{ width, height } | null
`deviceScaleFactor`	1, 2, 3
`isMobile / hasTouch`	boolean
`colorScheme`	'light' | 'dark' | 'no-preference'
`reducedMotion`	'reduce' | 'no-preference'
`forcedColors / contrast`	'active' / 'more'

POPULAR DEVICES

NAME	ENGINE
`Desktop Chrome / Edge`	chromium
`Desktop Safari`	webkit
`iPhone 15 / 15 Pro Max`	webkit
`Pixel 7`	chromium
`iPad Pro 11`	webkit

⚠ COMMON MISTAKES

- Expecting `devices['iPhone 15']` to behave exactly like iOS Safari; it's WebKit on desktop OS
- Setting `geolocation` without granting the permission; the page sees a denial
- Using `viewport: null` with device profiles; it disables the emulated size

✓ TIPS & BEST PRACTICES

- List names with `Object.keys(devices)`
- Test `reducedMotion: 'reduce'` to catch animation-dependent UI
- Use `locale` / `timezoneId` to reproduce date bugs users report

QUICK SUMMARY

- `devices[...]` — phone/tablet/desktop profiles
- `locale / timezoneId` — i18n and dates
- `colorScheme / reducedMotion` — media features
- `emulateMedia()` — switch at runtime

Clock, Permissions & Storage

CONTROLLING TIME, PERMISSIONS AND STORAGE

`page.clock` fakes `Date`, timers and `requestAnimationFrame`, so you can test countdowns, session expiry and "last seen 5 minutes ago" without waiting. Permissions such as notifications, clipboard and camera are granted per context. `addInitScript()` runs before any page script, which is the standard way to seed localStorage, feature flags or stub globals.

NOTES

- Call `clock.install()` **before** `goto` so the page sees fake time
- `setFixedTime` freezes `Date.now()` but timers still run
- `page.localStorage` / `sessionStorage` APIs added in 1.61
- `context.clearPermissions()` resets grants

EXAMPLES

```ts
// fake time
await page.clock.install({ time: new Date('2026-01-01T09:00:00') });
await page.goto('/session');
await page.clock.fastForward('29:00');                 // jump 29 minutes
await expect(page.getByText('Session expires in 1 minute')).toBeVisible();
await page.clock.runFor(60_000);                       // run timers 1 min
await expect(page.getByText('Session expired')).toBeVisible();

await page.clock.setFixedTime(new Date('2026-12-25T10:00:00'));   // frozen Date
await page.clock.pauseAt(new Date('2026-12-25T10:05:00'));

// permissions
await context.grantPermissions(['notifications', 'clipboard-read'],
  { origin: 'https://app.test' });

// seed storage before any app code runs
await page.addInitScript(() => {
  localStorage.setItem('onboarding-done', 'true');
  (window as any).__FEATURE_FLAGS__ = { newCheckout: true };
});
await page.goto('/');
```

CODE EXPLAINED

- → `install({ time })` — take over Date & timers
- → `fastForward` — jump, firing due timers once
- → `runFor` — run all timers in the period
- → `setFixedTime` — freeze Date only
- → `grantPermissions` — per origin or all
- → `addInitScript` — runs before page scripts

CLOCK API

METHOD	EFFECT
`install({ time })`	Fake Date + timers
`setFixedTime(t)`	Freeze Date.now()
`setSystemTime(t)`	Change wall clock only
`fastForward(ms | 'mm:ss')`	Jump ahead
`runFor(ms)`	Run timers for period
`pauseAt(t) / resume()`	Pause / continue

COMMON PERMISSIONS

NAME	GRANTS
`geolocation`	navigator.geolocation
`notifications`	Notification API
`clipboard-read/write`	navigator.clipboard
`camera / microphone`	getUserMedia
`background-sync`	Background Sync API

⚠ COMMON MISTAKES

- Installing the clock after `goto`; the page already captured real timers
- Using `setFixedTime` and expecting `setTimeout` callbacks to be frozen too
- Seeding localStorage with `page.evaluate` after load; the app already read the old value

✓ TIPS & BEST PRACTICES

- Combine `clock.install` with `pauseAt` to snapshot UIs at exact times
- Grant clipboard permissions to test copy buttons, then read with `page.evaluate`
- Headless clipboard is isolated since 1.62, so tests can't leak into each other

QUICK SUMMARY

- `page.clock` — deterministic time
- `fastForward / runFor` — advance time
- `grantPermissions()` — browser permissions
- `addInitScript()` — seed state before load

Visual Comparisons & Snapshots

SCREENSHOT TESTING

`await expect(page).toHaveScreenshot()` compares a screenshot with a stored **baseline**. The first run writes the baseline and fails (by design); later runs diff pixel-by-pixel. Playwright waits for two identical consecutive screenshots, disables animations and hides the caret to reduce noise. Baselines are stored **per browser and per OS**, because font rendering differs.

NOTES

- Baseline path: `<file>-snapshots/<name>-<project>-<platform>.png`
- WebP baselines supported (1.62/1.64)
- `--update-snapshots=missing` now passes (1.64)
- `toMatchSnapshot()` compares arbitrary text/buffers

EXAMPLES

```ts
await expect(page).toHaveScreenshot();                          // auto-named
await expect(page).toHaveScreenshot('landing.png', { fullPage: true });
await expect(page.getByTestId('pricing-card')).toHaveScreenshot('card.png');

await expect(page).toHaveScreenshot({
  maxDiffPixels: 100,                          // or maxDiffPixelRatio: 0.01
  threshold: 0.2,                              // per-pixel color tolerance 0-1
  mask: [page.getByTestId('clock'), page.locator('.ad')],   // pink boxes
  animations: 'disabled',
  stylePath: './tests/screenshot.css',         // hide dynamic bits via CSS
});

// playwright.config.ts
expect: { toHaveScreenshot: { maxDiffPixels: 50, type: 'webp' } },
snapshotPathTemplate: '{testDir}/__screenshots__/{testFilePath}/{arg}{ext}',
```

CODE EXPLAINED

- → no name — derived from test title
- → locator screenshot — element only
- → `maxDiffPixels` — tolerated changed pixels
- → `mask` — cover dynamic regions
- → `stylePath` — inject CSS for the shot
- → `snapshotPathTemplate` — custom layout

UPDATING BASELINES

COMMAND	EFFECT
`--update-snapshots`	Same as =changed
`=all`	Rewrite every baseline
`=changed`	Only mismatches + new
`=missing`	Only create new
`=none`	Never write

STABLE SCREENSHOTS

PROBLEM	FIX
`Fonts differ by OS`	Generate in Docker image
`Timestamps / ads`	mask or stylePath
`Animations`	animations: 'disabled'
`Lazy images`	Scroll / wait before shot
`Random data`	Mock APIs or seed data

⚠ COMMON MISTAKES

- Committing baselines made on macOS and running CI on Linux; every test fails
- Full-page screenshots of pages with live data; mask or mock it
- Blindly running `--update-snapshots` after failures; review the diff in the HTML report first

✓ TIPS & BEST PRACTICES

- Generate and update baselines inside the official Playwright Docker image
- Keep visual tests in their own project with one browser
- Prefer element screenshots of components over whole pages

QUICK SUMMARY

- `toHaveScreenshot()` — visual regression
- `maxDiffPixels / threshold` — tolerance
- `mask / stylePath` — hide noise
- `--update-snapshots` — refresh baselines

Accessibility & ARIA Snapshots

AUTOMATED ACCESSIBILITY CHECKS

Two complementary tools: **axe-core** (via `@axe-core/playwright`) scans the page for WCAG rule violations such as missing labels and low contrast, and **ARIA snapshots** assert the accessibility tree's structure as YAML. Axe finds rule breaks; ARIA snapshots catch regressions in what screen readers announce.

NOTES

- `npm i -D @axe-core/playwright`
- Automated tools catch roughly a third of issues; still test manually
- `page.ariaSnapshot()` and `toMatchAriaSnapshot` on page (1.60)
- Snapshot text supports RegExp: `- heading /Order #\d+/`

EXAMPLES

```ts
import AxeBuilder from '@axe-core/playwright';

test('checkout has no WCAG A/AA violations', async ({ page }) => {
  await page.goto('/checkout');
  const results = await new AxeBuilder({ page })
    .withTags(['wcag2a', 'wcag2aa', 'wcag21aa'])
    .exclude('#third-party-chat')
    .analyze();
  expect(results.violations).toEqual([]);
});

test('nav structure', async ({ page }) => {
  await page.goto('/');
  await expect(page.getByRole('navigation')).toMatchAriaSnapshot(`
    - navigation:
      - link "Home"
      - link "Pricing"
      - link /Docs/:
        - /url: /docs
  `);
});
```

CODE EXPLAINED

- → `AxeBuilder({ page })` — scan current page
- → `withTags` — limit to WCAG levels
- → `exclude` — skip regions you don't own
- → `violations` — array of failed rules
- → `toMatchAriaSnapshot` — YAML template of roles + names

ARIA SNAPSHOT SYNTAX

LINE	MATCHES
`- heading "Title" [level=1]`	Role, name, attribute
`- link /Docs/`	RegExp name
`- checkbox [checked]`	State
`- /url: /docs`	Link target property

AXEBUILDER METHODS

METHOD	PURPOSE
`include(sel) / exclude(sel)`	Scope
`withTags([...])`	Rule tags
`withRules([...])`	Only these rules
`disableRules([...])`	Skip rules
`analyze()`	Run, return results

⚠ COMMON MISTAKES

- Expecting `violations` to be `[]` on a legacy app; baseline known issues with `disableRules` first
- Scanning before the page renders; wait for a key element first
- Over-specifying ARIA snapshots; include only the structure that matters

✓ TIPS & BEST PRACTICES

- Wrap AxeBuilder in a fixture with your team's tags and exclusions
- Attach `results` JSON with `testInfo.attach` for triage
- Generate ARIA templates with codegen's "Assert snapshot" or `--update-snapshots`

QUICK SUMMARY

- `AxeBuilder.analyze()` — WCAG rule scan
- `withTags` — choose standards
- `toMatchAriaSnapshot` — tree regression check
- `ariaSnapshot()` — dump the tree

playwright.config.ts

THE CONFIG FILE

`playwright.config.ts` exports `defineConfig({...})`. Top-level keys control the **runner** (where tests are, workers, retries, reporters); `use` sets **browser/context options** inherited by every test; `projects` define named variants (browsers, devices, environments) that each run the matching tests with their own `use`. Project values override top-level ones.

NOTES

- `testProject.default: false` (1.64) keeps a project out of plain runs
- Use `process.env` for environment switches
- `--config other.config.ts` picks another file
- `dotenv` is not built in; import it yourself

TYPICAL CONFIG

```ts
import { defineConfig, devices } from '@playwright/test';
import 'dotenv/config';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 4 : undefined,
  reporter: process.env.CI ? [['blob'], ['github']] : 'html',
  use: {
    baseURL: process.env.BASE_URL ?? 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },
  projects: [
    { name: 'setup', testMatch: /.*\.setup\.ts/ },
    { name: 'chromium', use: { ...devices['Desktop Chrome'] }, dependencies: ['setup'] },
    { name: 'visual', testMatch: /visual\/.*/, default: false },   // only via --project
  ],
  webServer: { command: 'npm start', url: 'http://localhost:3000' },
});
```

CODE EXPLAINED

- → `fullyParallel` — tests within files run in parallel
- → `forbidOnly` — fail CI on stray `test.only`
- → `reporter` — different locally vs CI
- → `use` — defaults for every project
- → `default: false` — opt-in project (1.64)
- → `webServer` — start the app first

TOP-LEVEL KEYS

KEY	PURPOSE
`testDir / testMatch / testIgnore`	Which files
`timeout / globalTimeout`	Time limits
`workers / fullyParallel`	Parallelism
`retries / retryStrategy`	Flake handling
`reporter`	Output formats
`outputDir`	Artifacts folder

COMMON USE OPTIONS

OPTION	EXAMPLE
`baseURL`	'http://localhost:3000'
`trace`	'on-first-retry'
`screenshot`	'only-on-failure'
`video`	'retain-on-failure'
`headless`	true
`storageState`	'.auth/user.json'

⚠ COMMON MISTAKES

- Putting `baseURL` at the top level instead of inside `use`; it's ignored
- Defining `use` on a project and losing top-level `use` values you expected to merge deeply
- Hard-coding CI-only values (`workers: 1`) for local runs too

✓ TIPS & BEST PRACTICES

- Branch on `process.env.CI` for retries, workers and reporters
- Keep one config; use projects instead of several config files where possible
- Type-check config with `defineConfig` for autocompletion

QUICK SUMMARY

- `defineConfig` — typed config
- `use` — context/browser defaults
- `projects` — named variants
- `webServer` — boot the app under test

Parallelism & Sharding

RUNNING TESTS IN PARALLEL

Playwright runs test **files** in parallel across **worker** processes (default: half the CPU cores). With `fullyParallel: true`, individual tests in the same file are spread across workers too. **Sharding** splits the suite across machines with `--shard=x/n`; each shard writes a **blob report** that `merge-reports` combines into one HTML report.

NOTES

- Each worker is a separate Node process with its own browser
- Workers are restarted after a test failure
- `test.info().workerIndex` / `parallelIndex` identify workers
- `mode: 'serial'` skips remaining tests after a failure

CONFIGURATION AND COMMANDS

```ts
export default defineConfig({
  fullyParallel: true,
  workers: process.env.CI ? '50%' : undefined,      // number or % of cores
  reporter: process.env.CI ? 'blob' : 'html',
});

// per file
test.describe.configure({ mode: 'parallel' });     // tests in this file in parallel
test.describe.configure({ mode: 'serial' });       // dependent steps, in order

// shard across 4 CI machines
// npx playwright test --shard=1/4    (…2/4, 3/4, 4/4)
// → each writes blob-report/report-*.zip

// combine on one machine
// npx playwright merge-reports --reporter html ./all-blob-reports
// html reporter option (1.62): [['html', { mergeFiles: true }]]
```

CODE EXPLAINED

- → `fullyParallel` — parallelize inside files
- → `workers` — count or percentage
- → `mode: 'serial'` — ordered, shared failure
- → `--shard=i/n` — split run across machines
- → `merge-reports` — one report from all shards

DESCRIBE.CONFIGURE MODES

MODE	BEHAVIOUR
`default`	Sequential in file, independent retries
`parallel`	Tests in file run concurrently
`serial`	In order; on failure skip the rest; retry group

CLI FLAGS

FLAG	EFFECT
`--workers=4 / -j 4`	Worker count
`--workers=1`	Fully serial run
`--shard=2/5`	Run shard 2 of 5
`--fully-parallel`	Force fullyParallel
`--max-failures=10 / -x`	Stop early

⚠ COMMON MISTAKES

- Tests sharing a user account or DB rows; they pass alone and fail in parallel
- Using `mode: 'serial'` to fix shared state; it hides the coupling and slows the suite
- Forgetting `fullyParallel` and wondering why shards are unbalanced (sharding splits by file otherwise)

✓ TIPS & BEST PRACTICES

- Use unique data per test (`Date.now()` , worker index) to stay parallel-safe
- Enable `fullyParallel` before sharding for even shard sizes
- Upload `blob-report` as a CI artifact from every shard, then merge

QUICK SUMMARY

- `workers` — processes per machine
- `fullyParallel` — per-test distribution
- `--shard=i/n` — split across machines
- `merge-reports` — combine blob reports

Retries & Flaky Tests

HANDLING FLAKINESS

With `retries > 0`, a failed test is re-run in a fresh worker. A test that fails first and then passes is reported as **flaky**: green, but flagged. `retryStrategy` (1.62) chooses whether retries run immediately or **isolated** at the end, and `failOnFlakyTests` turns flaky results into a failing exit code. `--repeat-each` and `--shuffle` (1.64) help reproduce order- and timing-dependent bugs.

NOTES

- Retries restart the worker; `beforeAll` runs again
- Serial groups retry from the start of the group
- `testInfo.retry` is 0 on the first attempt
- Traces with `'on-first-retry'` capture the failing rerun

CONFIGURATION

```ts
export default defineConfig({
  retries: process.env.CI ? 2 : 0,
  retryStrategy: 'isolated',          // retries run last, one by one (1.62)
  failOnFlakyTests: !!process.env.CI, // flaky = exit code 1
  use: { trace: 'on-first-retry', video: 'retain-on-failure-and-retries' },
});

test('dashboard loads', async ({ page }, testInfo) => {
  if (testInfo.retry) await page.context().clearCookies();   // extra cleanup on retry
  await page.goto('/dashboard');
});

test.describe.configure({ retries: 3 });          // per-file override

// reproduce flakiness locally
// npx playwright test cart.spec.ts --repeat-each=20 --workers=4
// npx playwright test --shuffle                 (random order, 1.64)
// npx playwright test --shuffle=1234            (replay an order)
```

CODE EXPLAINED

- → `retries` — attempts after the first
- → `retryStrategy: 'isolated'` — less interference
- → `failOnFlakyTests` — don't let flakes pass CI
- → `testInfo.retry` — branch on retry attempt
- → `--repeat-each` — stress one test
- → `--shuffle[=seed]` — find order dependencies

RESULT STATUS

STATUS	MEANING
`passed`	Passed first time
`flaky`	Failed, then passed on retry
`failed`	Failed every attempt
`skipped`	Not run
`timedOut`	Exceeded timeout

COMMON FLAKE CAUSES

CAUSE	FIX
`Fixed sleeps`	Web-first assertions
`Shared test data`	Unique data per test
`Animations`	reducedMotion / disable
`Order dependence`	--shuffle, isolate state
`Slow CI`	Bigger timeouts on slow paths

⚠ COMMON MISTAKES

- Treating retries as a fix; flaky tests still hide real race conditions
- Using `retries` without traces; you can't see what went wrong in the failed attempt
- Raising timeouts everywhere instead of finding the one slow wait

✓ TIPS & BEST PRACTICES

- Track flaky tests from the HTML/JSON report and fix the top offenders weekly
- Use `--repeat-each=20` locally before declaring a test fixed
- Set `video: 'retain-on-failure-and-retries'` (1.61) to see both attempts

QUICK SUMMARY

- `retries` — re-run failures
- `flaky` — passed after retry
- `failOnFlakyTests` — strict CI
- `--repeat-each / --shuffle` — reproduce flakes

Reporters

REPORTING RESULTS

Reporters turn test events into output. Terminal reporters (`list`, `line`, `dot`) show progress; file reporters (`html`, `json`, `junit`, `blob`) feed CI and dashboards. You can use several at once. The **HTML report** includes traces, screenshots, videos and attachments, and is opened with `npx playwright show-report`. Custom reporters implement the `Reporter` interface.

NOTES

- Default: `list` locally, `dot` on CI
- `--add-reporter` (1.63) adds to config reporters instead of replacing
- HTML `open`: `'always' | 'never' | 'on-failure'`
- `PLAYWRIGHT_JUNIT_OUTPUT_FILE` / `PLAYWRIGHT_JSON_OUTPUT_FILE` set paths

CONFIGURATION

```ts
export default defineConfig({
  reporter: [
    ['list'],
    ['html', { open: 'never', outputFolder: 'playwright-report' }],
    ['junit', { outputFile: 'results/junit.xml' }],
    ['./reporters/slack-reporter.ts', { channel: '#qa' }],
  ],
});

// reporters/slack-reporter.ts
import type { Reporter, TestCase, TestResult, FullResult }
  from '@playwright/test/reporter';
export default class SlackReporter implements Reporter {
  failed = 0;
  onTestEnd(test: TestCase, r: TestResult) { if (r.status === 'failed') this.failed++; }
  async onEnd(r: FullResult) { console.log(`Run ${r.status}: ${this.failed} failed`); }
}
// npx playwright test --reporter=dot --add-reporter=junit
```

CODE EXPLAINED

- → array form — several reporters at once
- → options per reporter — paths, open behaviour
- → path string — your own reporter module
- → `onTestEnd` / `onEnd` — reporter hooks

BUILT-IN REPORTERS

NAME	OUTPUT
`list`	One line per test (local default)
`line`	Single updating line
`dot`	One char per test (CI default)
`html`	Interactive report folder
`json / junit`	Machine-readable files
`blob`	Shard data for merge-reports
`github`	Annotations in GitHub Actions
`perfetto`	Timeline trace (1.63)

REPORTER HOOKS

HOOK	CALLED
`onBegin(config, suite)`	Run starts
`onTestBegin(test)`	Test starts
`onStepBegin/End`	Each step
`onTestEnd(test, result)`	Test finishes
`onError(error)`	Global error
`onEnd(result)`	Run finishes

⚠ COMMON MISTAKES

- Using `--reporter=junit` and losing the HTML report from config; use `--add-reporter`
- Leaving HTML `open` at default on CI; it tries to serve the report and blocks
- Writing JUnit to a path your CI doesn't collect

✓ TIPS & BEST PRACTICES

- On CI use `blob` + `github`, merge to `html` in a final job
- Upload `playwright-report/` as an artifact; it's self-contained
- Use `omitTags` / `title` options to keep reports readable

QUICK SUMMARY

- `list / dot / line` — terminal
- `html` — rich interactive report
- `junit / json / blob` — CI integration
- `Reporter` interface — custom output

CLI Filtering & Flags

file:line · --grep · --grep-invert · --project · --last-failed · --only-changed · --list · --max-failures

CHOOSING WHAT RUNS

The test command combines several filters: **file filters** (positional regex on paths, optionally `:line`), **title filters** (`--grep`/`-g`, `--grep-invert`/`-G`), **project filters** (`--project`, repeatable, globs allowed) and **change-based filters** (`--last-failed`, `--only-changed`). Everything a filter selects still respects `test.only` and `skip` annotations.

NOTES

- Grep matches the full title path incl. project, file and tags
- `--only-changed[=ref]` uses git to pick affected test files
- `--last-failed` reads `test-results/.last-run.json`
- `--list` prints what would run without running it

RECIPES

```
npx playwright test tests/cart                      # files under tests/cart
npx playwright test cart.spec.ts:42                 # one test by line
npx playwright test cart.spec.ts:42:5               # line + column
npx playwright test -g "applies coupon"             # title contains
npx playwright test -g "@smoke" -G "@slow"          # tag in, tag out
npx playwright test --project=chromium --project=firefox
npx playwright test --project="mobile-*"            # glob on project names
npx playwright test --last-failed                   # only last failures
npx playwright test --only-changed=origin/main      # files touched vs main
npx playwright test --list -g "@smoke"              # dry run
npx playwright test -x                              # stop at first failure
npx playwright test --max-failures=5
npx playwright test --repeat-each=10 --workers=1
npx playwright test --pass-with-no-tests            # don't fail empty runs
npx playwright test --tsconfig=tsconfig.e2e.json
```

CODE EXPLAINED

- → positional — regex on file path
- → `:line[:col]` — exact test or describe
- → `-g` / `-G` — include / exclude titles
- → `--project` — name or glob, repeatable
- → `--only-changed` — git-aware selection
- → `-x` — fail fast

SELECTION FLAGS

FLAG	SELECTS
`--grep / -g`	Titles matching regex
`--grep-invert / -G`	Titles not matching
`--project`	Named projects
`--last-failed`	Previous failures
`--only-changed`	Git-changed files
`--no-deps`	Skip dependency projects

RUN-CONTROL FLAGS

FLAG	EFFECT
`--headed`	Show browsers
`--debug`	Inspector, 1 worker
`--ui`	UI mode
`--trace on`	Force tracing
`--quiet`	Hide stdio from tests
`--update-snapshots`	Rewrite baselines
`--ignore-snapshots`	Skip visual asserts
`--forbid-only`	Fail on test.only

⚠ COMMON MISTAKES

- `npx playwright test "applies coupon"`: positional args filter **paths**, not titles
- Unquoted globs (`--project=mobile-*`) expanded by the shell
- Expecting `--last-failed` to work after cleaning `test-results/`

✓ TIPS & BEST PRACTICES

- Put `--only-changed=origin/main` in a fast pre-merge CI job
- Use `--list` to check tag filters before a long run
- Combine `-x` with `--headed` while fixing one broken flow

QUICK SUMMARY

- `path[:line]` — file / test selection
- `-g / -G` — title include / exclude
- `--project` — choose projects
- `--last-failed / --only-changed` — smart reruns

UI Mode

INTERACTIVE UI MODE

`npx playwright test --ui` opens a desktop-style app for exploring, running and debugging tests. Every action is captured like a trace: a **timeline** of screenshots, DOM snapshots you can inspect, network, console, source and errors. **Watch mode** re-runs a test whenever its file changes, making UI mode the most productive way to write tests.

NOTES

- Uses the same config and projects as the CLI
- `--ui-host=0.0.0.0 --ui-port=8080` serves it to a browser (containers, remote)
- Filter by text, `@tag`, status and project in the sidebar
- Pick locator → click an element in a snapshot to get its locator

PANELS

PANEL	SHOWS
`Sidebar`	Test tree, filters, run/watch toggles per test
`Timeline`	Filmstrip of the run; hover to scrub
`Actions`	Each step with before/after DOM snapshots
`Snapshot`	Interactive DOM; open in new window, pick locator
`Source`	Test source with the current line highlighted
`Call`	Arguments, timing, resolved locator
`Log`	Actionability log: waits and retries
`Errors`	Failure messages and stack
`Console / Network`	Browser logs and requests
`Attachments`	Screenshots, files from testInfo.attach

LAUNCHING

```
npx playwright test --ui                          # open UI mode
npx playwright test --ui tests/cart               # pre-filtered
npx playwright test --ui --project=chromium
npx playwright test --ui-host=0.0.0.0 --ui-port=9323   # browser access
# Docker: expose 9323 and open http://localhost:9323
```

TIPS FOR USE

- → Eye icon — toggle watch mode per test or file
- → Snapshots are live DOM: use DevTools on them
- → Locator field — type a locator to highlight matches
- → Run a single test repeatedly while editing

⚠ COMMON MISTAKES

- Debugging failures only from terminal output when UI mode shows the DOM at each step
- Leaving watch on for the whole suite; every save re-runs everything
- Forgetting UI mode respects `fullyParallel` /workers; heavy suites still use many cores

✓ TIPS & BEST PRACTICES

- Write new tests in UI mode with watch on for instant feedback
- Use the Pick Locator tool to settle selector choices
- Open the snapshot in a new window to use browser DevTools on it

QUICK SUMMARY

- `--ui` — interactive runner
- watch mode — rerun on save
- timeline + snapshots — time-travel debugging
- `--ui-host/--ui-port` — remote access

Trace Viewer

POST-MORTEM DEBUGGING WITH TRACES

A **trace** is a zip file holding every action, DOM snapshots before and after, screenshots, network (including WebSockets since 1.61), console logs and source. Record traces in CI and open them with `npx playwright show-trace`, from the HTML report, or by dropping the file onto **trace.playwright.dev** (runs locally in your browser). Since 1.59, `npx playwright trace` offers CLI subcommands for agents and scripts.

NOTES

- `'on-first-retry'` is the recommended CI setting
- Traces add overhead; avoid `'on'` for the full suite
- `snapshots: { dom, aria, screen }` controls captured detail (1.63)
- Library mode: `context.tracing.start()` / `stop({ path })`

RECORDING AND OPENING

```
// playwright.config.ts
use: { trace: 'on-first-retry' },              // or 'retain-on-failure'

// CLI override for one run
// npx playwright test --trace on

// library / manual
await context.tracing.start({ screenshots: true, snapshots: true, sources: true });
await page.goto('/');
await context.tracing.stop({ path: 'trace.zip' });

// opening
// npx playwright show-trace test-results/cart-chromium-retry1/trace.zip
// npx playwright show-report   → click the trace icon
// npx playwright trace open trace.zip       (1.59 CLI subcommands)
// npx playwright trace actions              → list actions
// npx playwright trace snapshot <action-id> → dump a DOM snapshot
```

CODE EXPLAINED

- → `trace` option — when to record
- → `--trace on` — force for one run
- → `tracing.start/stop` — manual control
- → `show-trace` — open viewer
- → `trace …` subcommands — scriptable access

TRACE MODES

MODE	RECORDS
`off`	Nothing
`on`	Every test (slow)
`on-first-retry`	Only the first retry
`on-all-retries`	Every retry
`retain-on-failure`	All, keep failures only
`retain-on-first-failure`	Keep first failure only

VIEWER TABS

TAB	CONTENT
`Actions`	Steps with before/after snapshot
`Metadata`	Browser, viewport, duration
`Call / Log`	Arguments and auto-wait log
`Network`	Requests, timing, bodies
`Console`	Logs and page errors
`Attachments`	Screenshots, files

⚠ COMMON MISTAKES

- `trace: 'on'` in CI for thousands of tests; huge artifacts and slower runs
- Uploading traces with real user data to public places; traces contain everything the page saw
- Looking only at the final screenshot; the "before" snapshot of the failing action usually shows the cause

✓ TIPS & BEST PRACTICES

- Upload `test-results/` as a CI artifact so traces survive the job
- Use the Log tab to see what auto-wait was waiting for
- Share traces via trace.playwright.dev; nothing is uploaded to a server

QUICK SUMMARY

- `trace: 'on-first-retry'` — CI default
- `show-trace` / HTML report — open traces
- trace.playwright.dev — browser-only viewer
- `npx playwright trace` — CLI access

Inspector & Debug Mode

LIVE DEBUGGING

`--debug` runs headed with one worker, no timeout, and opens the **Playwright Inspector**, which pauses before each action so you can step, see the target highlighted, and edit locators live. `page.pause()` stops at a specific line. Since 1.59, `--debug=cli` exposes the paused session to `playwright-cli` so a coding agent can inspect it.

NOTES

- `PWDEBUG=1` = `--debug` for any runner (Python, Java, .NET)
- `PWDEBUG=console` exposes `playwright` object in DevTools
- IDE breakpoints work with `--debug` or VS Code debugging
- `DEBUG=pw:api` logs every API call to stderr

DEBUGGING TOOLS

```
// run one test under the Inspector
// npx playwright test cart.spec.ts:18 --debug
// PWDEBUG=1 npx playwright test          (same, via env var)
// npx playwright test --debug=cli        (agent-driven, 1.59)

test('checkout', async ({ page }) => {
  await page.goto('/cart');
  await page.pause();                                  // opens Inspector here
  await page.getByRole('button', { name: 'Checkout' }).highlight();   // visual check
  console.log(await page.getByRole('row').count());     // quick probe
});

// verbose logs
// DEBUG=pw:api npx playwright test                   API calls
// DEBUG=pw:browser npx playwright test               browser stdout
// in DevTools console with PWDEBUG=console:
//   playwright.$('text=Checkout')  playwright.inspect(sel)  playwright.locator(sel)
```

CODE EXPLAINED

- → `--debug` — headed, step-through, no timeout
- → `page.pause()` — breakpoint in the browser
- → `highlight()` — outline what a locator matches
- → `DEBUG=pw:api` — log calls and waits
- → `PWDEBUG=console` — DevTools helpers

INSPECTOR CONTROLS

CONTROL	DOES
`Resume (F8)`	Run to next pause
`Step over (F10)`	Run one action
`Pick locator`	Click page → locator
`Locator field`	Edit + highlight live
`Record`	Append recorded actions

DEBUG ENV VARS

VARIABLE	EFFECT
`PWDEBUG=1`	Inspector mode
`PWDEBUG=console`	DevTools helpers
`DEBUG=pw:api`	API call log
`DEBUG=pw:browser`	Browser logs
`PLAYWRIGHT_FORCE_TTY`	Force TTY output

⚠ COMMON MISTAKES

- Leaving `page.pause()` in committed code; CI hangs until the test timeout
- Debugging with many workers; use `--debug` (forces one worker) or `--workers=1`
- Reading only the error message; the Inspector's log shows each actionability retry

✓ TIPS & BEST PRACTICES

- Combine `cart.spec.ts:18 --debug` to start at exactly one test
- Use `highlight()` while tuning locators in headed mode
- Use `DEBUG=pw:api` in CI to see the last action before a hang

QUICK SUMMARY

- `--debug` / `PWDEBUG=1` — Inspector
- `page.pause()` — targeted breakpoint
- `highlight()` — visualize locators
- `DEBUG=pw:api` — verbose logging

Codegen

RECORDING TESTS

`npx playwright codegen <url>` opens a browser plus the Inspector and writes code as you click and type. It picks resilient locators (role, label, text, test id), and the toolbar adds **assertions** (visible, text, value, ARIA snapshot) by clicking elements. Output can target any supported language and runner. Since 1.59, `page.pickLocator()` lets scripts and agents request a locator interactively.

NOTES

- Generated code is a starting point; refactor into page objects
- Codegen honours `testIdAttribute` via `--test-id-attribute`
- `--save-storage` records login state for reuse
- Recording inside an existing test: VS Code "Record at cursor"

COMMANDS

```
npx playwright codegen                                    # blank page
npx playwright codegen http://localhost:3000
npx playwright codegen --target=python-pytest demo.playwright.dev/todomvc
npx playwright codegen -o tests/recorded.spec.ts localhost:3000
npx playwright codegen --device="iPhone 15" localhost:3000
npx playwright codegen --viewport-size="1280,720" --color-scheme=dark localhost:3000
npx playwright codegen --lang="de-DE" --timezone="Europe/Berlin" localhost:3000
npx playwright codegen --geolocation="41.89,12.49" localhost:3000
npx playwright codegen --save-storage=auth.json localhost:3000/login   # log in, close
npx playwright codegen --load-storage=auth.json localhost:3000         # start logged in
npx playwright codegen --test-id-attribute=data-qa localhost:3000
npx playwright codegen --http-credentials=user:pass intranet.local     # 1.63
```

CODE EXPLAINED

- → `--target` — language/runner of output
- → `-o` — write to a file
- → `--device` / emulation flags — record mobile, locale, dark
- → `--save-storage` — capture auth state
- → `--load-storage` — reuse it

--TARGET VALUES

VALUE	OUTPUT
`playwright-test`	TS/JS @playwright/test (default)
`javascript`	Library script
`python-pytest / python / python-async`	Python
`java / java-junit`	Java
`csharp-nunit / csharp-mstest / csharp`	.NET

INSPECTOR TOOLBAR

BUTTON	ADDS
`Record`	Start/stop recording
`Pick locator`	Copy a locator
`Assert visibility`	toBeVisible()
`Assert text / value`	toContainText / toHaveValue
`Assert snapshot`	toMatchAriaSnapshot

⚠ COMMON MISTAKES

- Committing raw recordings full of `nth()` and duplicated steps
- Recording against production with real accounts and saving the storage file in git
- Expecting codegen to record waits; it relies on auto-waiting, add assertions yourself

✓ TIPS & BEST PRACTICES

- Record the happy path, then add edge cases by hand
- Use the assertion buttons; recordings without assertions test nothing
- Use codegen just to discover locators for tricky elements

QUICK SUMMARY

- `codegen <url>` — record actions
- `--target` — choose language
- assert buttons — add expectations
- `--save/--load-storage` — auth for recording

Screenshots, Video & Screencast

CAPTURING MEDIA

The runner can attach a **screenshot** and a **video** of each test automatically, controlled by `use.screenshot` and `use.video`. Manual `page.screenshot()` / `locator.screenshot()` capture images on demand (PNG, JPEG or WebP). The `page.screencast` API (1.59) records polished demo videos with action annotations, chapter titles and overlays, and can stream frames.

NOTES

- Videos are written when the context closes
- `video.fps` option added in 1.64; `fontSize` deprecated
- WebP screenshots: `.webp` names + `quality` (1.62)
- Library mode: `browser.newContext({ recordVideo: { dir } })`

EXAMPLES

```ts
// playwright.config.ts
use: {
  screenshot: 'only-on-failure',                // 'on' | 'off'
  video: { mode: 'retain-on-failure', size: { width: 1280, height: 720 }, fps: 15 },
},

// manual screenshots
await page.screenshot({ path: 'shots/home.png', fullPage: true });
await page.screenshot({ path: 'shots/home.webp', quality: 80 });
await page.getByTestId('chart').screenshot({ path: 'shots/chart.png' });
const buffer = await page.screenshot();          // in memory

// demo recording with annotations (1.59)
await page.screencast.start({ path: 'demo.webm', size: { width: 1280, height: 800 } });
await page.screencast.showChapter('Checkout', { description: 'Pay with card' });
await page.screencast.showActions({ position: 'bottom' });
await page.getByRole('button', { name: 'Pay' }).click();
await page.screencast.stop();
```

CODE EXPLAINED

- → `screenshot` option — automatic per test
- → `video.mode` — when to keep videos
- → `fullPage` — whole scrollable page
- → `locator.screenshot()` — one element
- → `screencast` — annotated demo videos

VIDEO MODES

MODE	KEEPS
`off` / `on`	None / all
`retain-on-failure`	Failed tests only
`on-first-retry`	First retry only
`on-all-retries`	Every retry (1.61)
`retain-on-first-failure`	First failure (1.61)
`retain-on-failure-and-retries`	Failures + retries (1.61)

SCREENSHOT() OPTIONS

OPTION	EFFECT
`path` / `type`	File + png/jpeg/webp
`fullPage`	Whole page
`clip`	Region { x, y, width, height }
`mask`	Cover locators
`omitBackground`	Transparent background
`animations`	'disabled' freezes them

⚠ COMMON MISTAKES

- `video: 'on'` for the whole suite in CI; large artifacts and slower runs
- Reading the video path before the context closes; the file isn't finished
- Using screenshots as assertions; use `toHaveScreenshot` for comparisons

✓ TIPS & BEST PRACTICES

- `retain-on-failure` gives videos only where they help
- Use `screencast` chapters to make product demo videos from tests
- Attach targeted screenshots with `testInfo.attach` at key steps

QUICK SUMMARY

- `use.screenshot` / `use.video` — automatic capture
- `page.screenshot()` — manual images
- `video.fps` — frame rate (1.64)
- `page.screencast` — annotated recordings

Test Agents

AI TEST AGENTS

Since 1.56, Playwright ships three **agent definitions** for AI coding tools: the **planner** explores your app and writes a Markdown test plan, the **generator** turns plans into Playwright tests while verifying locators live, and the **healer** runs failing tests, inspects the UI and patches or skips them. `npx playwright init-agents` installs the definitions for your agent loop.

NOTES

- Supported loops include VS Code (Copilot), Claude Code, Codex, OpenCode
- Re-run `init-agents` after upgrading Playwright
- Needs a **seed test** that sets up the app and provides `page`
- VS Code agent mode needs v1.105+

SETUP AND FLOW

```
npx playwright init-agents --loop=vscode      # GitHub Copilot in VS Code
npx playwright init-agents --loop=claude      # Claude Code

# project after init
# .github/ (or agent folder)  agent definitions
# specs/                      Markdown test plans (planner output)
# tests/seed.spec.ts          seed test: env setup + ready page
# tests/*.spec.ts             generated tests

# 1. "Plan tests for the guest checkout"  → planner → specs/guest-checkout.md
# 2. "Generate tests from specs/guest-checkout.md" → generator → tests/
# 3. "Fix failing tests"                  → healer → passing or test.fixme
```

CODE EXPLAINED

- → `--loop` — target agent tool
- → `specs/` — reviewable plans in Markdown
- → seed test — shared starting point
- → healer — repairs or marks broken features

TESTS/SEED.SPEC.TS

```
import { test, expect } from './fixtures';
test('seed', async ({ page }) => {
  await page.goto('/');                // agents start from this state
  await expect(page.getByRole('banner')).toBeVisible();
});
```

SEED TEST

- → Uses your fixtures (auth, data)
- → Keep it minimal and green

⚠ COMMON MISTAKES

- Accepting generated tests without review; agents can assert the wrong behaviour
- Skipping the seed test; the planner explores a logged-out or empty app
- Letting the healer "fix" a real product bug by weakening an assertion

✓ TIPS & BEST PRACTICES

- Review plans in `specs/` before generating; it's cheaper to fix a plan than tests
- Give the planner a PRD or user story for better coverage
- Run generated tests in CI like any other test; agents are authoring tools

QUICK SUMMARY

- `init-agents --loop=…` — install definitions
- planner — Markdown test plans
- generator — tests from plans
- healer — repair failing tests

Playwright MCP & playwright-cli

npx playwright mcp · @playwright/mcp · browser.bind() · npx playwright cli · attach / show · webmcp-call · init-skills

BROWSER AUTOMATION FOR AI AGENTS

Playwright MCP is a Model Context Protocol server that lets AI assistants drive a browser using accessibility-tree snapshots instead of screenshots. **playwright-cli** offers the same capabilities as shell commands, which is more token-efficient for coding agents. Both are bundled in Playwright (1.62) and can attach to a browser your tests opened via `browser.bind()` (1.59).

NOTES

- MCP: best for long, stateful exploratory loops
- CLI + skills: best for coding agents (Claude Code, Copilot)
- WebMCP tools on pages exposed as `webmcp_<tool>` (1.64)
- Treat page-provided WebMCP tools as untrusted input

SETUP AND COMMANDS

```
# MCP server — client config (Claude Desktop, VS Code, Cursor …)
# { "mcpServers": { "playwright": {
#     "command": "npx", "args": ["@playwright/mcp@latest"] } } }
claude mcp add playwright npx @playwright/mcp@latest      # Claude Code
npx playwright mcp --headless --browser=firefox            # bundled (1.62)

# playwright-cli
npm i -g @playwright/cli@latest        # or: npx playwright cli
playwright-cli install --skills        # agent skills for this project
playwright-cli open https://demo.playwright.dev/todomvc --headed
playwright-cli snapshot                # page tree with refs (e12 …)
playwright-cli fill e12 "Buy milk"
playwright-cli press Enter
playwright-cli screenshot
playwright-cli attach --extension      # your real browser tabs
playwright-cli show                    # dashboard of sessions
playwright-cli webmcp-list             # page-exposed tools (1.64)
```

CODE EXPLAINED

- → `@playwright/mcp` — MCP server package
- → `--headless` / `--browser` — server options
- → `snapshot` — returns element refs
- → `fill e12 …` — act on a ref
- → `attach` / `show` — existing browser, dashboard

MCP VS CLI

ASPECT	MCP · CLI
Interface	Tool calls · Shell commands
Token cost	Higher (schemas, trees) · Lower
State	Persistent session · Named sessions (-s=)
Best for	Exploration loops · Coding agents

COMMON MCP OPTIONS

OPTION	EFFECT
`--headless`	No window
`--browser`	chrome, firefox, webkit, msedge
`--isolated`	In-memory profile
`--storage-state`	Start logged in
`--caps`	Extra capabilities (pdf, vision)
`--no-webmcp`	Disable WebMCP tools

⚠ COMMON MISTAKES

- Giving an agent a browser profile logged in to production admin accounts
- Following instructions found on web pages the agent visits (prompt injection)
- Using MCP to "write tests" without saving them as real spec files

✓ TIPS & BEST PRACTICES

- Use `--isolated` or named CLI sessions to keep agent browsing separate
- Pair agents with test agents (`init-agents`) for test authoring
- Use `browser.bind()` to let an agent inspect a paused test's browser

QUICK SUMMARY

- `@playwright/mcp` — MCP server
- `playwright-cli` — token-efficient CLI
- `install --skills` — agent skills
- `attach / show` — reuse and monitor sessions

webServer & baseURL

webServer.command · url · reuseExistingServer · timeout · multiple servers · env · gracefulShutdown

STARTING YOUR APP FOR TESTS

The `webServer` config starts your app (or API, or mock server) before tests and stops it afterwards. Playwright waits until `url` responds (2xx–4xx) or `port` accepts connections. Combined with `use.baseURL`, tests call `page.goto('/path')` and work against any framework: React, Next.js, Vue, Angular or SvelteKit just differ in the start command and port.

NOTES

- `reuseExistingServer: !process.env.CI` reuses your local dev server
- Array form starts several servers (frontend + API)
- Default startup timeout: 60 s
- Without `gracefulShutdown` the process group is SIGKILLed

PLAYWRIGHT.CONFIG.TS

```ts
export default defineConfig({
  use: { baseURL: 'http://localhost:3000' },
  webServer: [
    {
      name: 'api',                                  // log prefix
      command: 'npm run start --workspace=api',
      url: 'http://localhost:4000/health',
      env: { DATABASE_URL: 'postgres://localhost/e2e' },
      reuseExistingServer: !process.env.CI,
    },
    {
      name: 'web',
      command: process.env.CI ? 'npm run build && npm run start' : 'npm run dev',
      url: 'http://localhost:3000',
      timeout: 120_000,                             // slow builds
      gracefulShutdown: { signal: 'SIGTERM', timeout: 5_000 },
      reuseExistingServer: !process.env.CI,
    },
  ],
});
```

CODE EXPLAINED

- → array — several servers, started in order
- → `url` — readiness check endpoint
- → `env` — server environment variables
- → CI vs local command — prod build on CI
- → `gracefulShutdown` — clean stop

WEBSERVER OPTIONS

OPTION	PURPOSE
`command`	Shell command to start
`url / port`	Readiness check (pick one)
`reuseExistingServer`	Use already-running server
`timeout`	Startup limit (60 000 ms)
`cwd / env`	Working dir / env vars
`wait`	Regex on output; groups → env
`name`	Log prefix

FRAMEWORK DEFAULTS

FRAMEWORK	DEV · PROD PREVIEW
`Vite (React/Vue)`	5173 · 4173
`Next.js / Nuxt`	3000 · 3000
`Angular`	4200 · (static)
`SvelteKit`	5173 · 4173
`Astro`	4321 · 4321

⚠ COMMON MISTAKES

- Testing against `npm run dev` in CI; dev servers compile on demand and make tests slow and flaky
- `url` pointing at a page that needs auth and returns 401 forever; use a health endpoint
- Hard-coding ports that differ between dev and preview

✓ TIPS & BEST PRACTICES

- Run the production build in CI (`build && start/preview`)
- Use a dedicated `.env.test` for the server via `env`
- Use `wait: { stdout: /listening on (?<PORT>\d+)/ }` for random ports

QUICK SUMMARY

- `webServer` — boot the app
- `url` — readiness probe
- `reuseExistingServer` — reuse local server
- `baseURL` — relative `goto()`

React (Vite) Apps

npm run dev vs preview · port 5173 · data-testid strategy · React Router · mocking fetch · StrictMode double render

TESTING REACT SPAS

A Vite-based React app is a static bundle talking to APIs, which makes it ideal for Playwright: build once, serve with `vite preview`, and **mock the API** with `page.route()` for fast, deterministic UI tests. Client-side routing means URL changes aren't page loads, so assert with `toHaveURL` and visible content. React Testing Library habits (roles, labels) translate directly to Playwright locators.

NOTES

- `npm create vite@latest app -- --template react-ts`
- Dev: 5173 · `vite preview` : 4173
- StrictMode double-invokes effects in dev only: duplicate requests
- `VITE_*` env vars are inlined at build time

CONFIG + TEST

```ts
// playwright.config.ts
export default defineConfig({
  use: { baseURL: 'http://localhost:4173' },
  webServer: {
    command: 'npm run build && npm run preview -- --port 4173 --strictPort',
    url: 'http://localhost:4173',
    reuseExistingServer: !process.env.CI,
  },
});

// tests/products.spec.ts
test('lists products from the API', async ({ page }) => {
  await page.route('**/api/products', (r) =>
    r.fulfill({ json: [{ id: 1, name: 'Mug', price: 9 }] }));
  await page.goto('/products');                          // React Router route
  await expect(page.getByRole('heading', { name: 'Products' })).toBeVisible();
  await page.getByRole('link', { name: 'Mug' }).click();
  await expect(page).toHaveURL('/products/1');           // client-side navigation
  await expect(page.getByText('$9.00')).toBeVisible();
});
```

CODE EXPLAINED

- → `build && preview` — test the production bundle
- → `--strictPort` — fail instead of picking another port
- → `page.route` — deterministic API data
- → `toHaveURL` — SPA route change

REACT GOTCHAS

ISSUE	APPROACH
`Suspense / lazy routes`	Assert on content, not timing
`Debounced inputs`	expect auto-retries; no sleeps
`Portals (modals, toasts)`	getByRole('dialog') at page level
`Virtualized lists`	Scroll, then locate
`Duplicate requests`	Dev StrictMode; test the build

TEST-ID STRATEGY

DO	WHY
`Prefer roles + labels`	Accessible + stable
`data-testid on icons/charts`	No text to target
`Strip test IDs? Usually no`	Cheap, stable contract

⚠ COMMON MISTAKES

- Testing against `vite dev` in CI; first-load module compilation slows tests
- Expecting `page.goto` to wait for React data fetching; assert the content instead
- Using `page.waitForLoadState('networkidle')` as a "React finished" signal

✓ TIPS & BEST PRACTICES

- Combine Playwright e2e with component tests (sheets 53–54) for UI states
- Mock only the backend boundary; keep routing and state real
- Use React Developer Tools in headed/debug mode to inspect state

QUICK SUMMARY

- `build && preview` — production-like server
- `page.route()` — mock the API
- `toHaveURL` — client routing
- roles first, `data-testid` for the rest

Next.js Apps

TESTING NEXT.JS

Next.js renders on the server, so `page.route()` only intercepts **browser** requests; server-side `fetch` in Server Components and Server Actions never reaches it. Test the real production server (`next build && next start`), seed data through APIs or the database, and use the `request` fixture for Route Handlers. Next's experimental **test mode** can mock server-side fetches when you need it.

NOTES

- `npx create-next-app@latest --example with-playwright`
- Server Actions are tested through the UI form that calls them
- `proxy.ts` (formerly middleware) runs on every request; test redirects with `toHaveURL`
- `NEXT_PUBLIC_*` vars are baked in at build time

CONFIG + TESTS

```ts
// playwright.config.ts
webServer: {
  command: 'npm run build && npm run start',
  url: 'http://localhost:3000',
  reuseExistingServer: !process.env.CI,
},

test('server action creates a post', async ({ page }) => {
  await page.goto('/posts/new');
  await page.getByLabel('Title').fill('Hello');
  await page.getByRole('button', { name: 'Publish' }).click();   // form action
  await expect(page).toHaveURL(/\/posts\/\d+/);                  // redirect()
  await expect(page.getByRole('heading', { name: 'Hello' })).toBeVisible();
});
test('route handler returns JSON', async ({ request }) => {
  const res = await request.get('/api/health');                  // app/api/health/route.ts
  await expect(res).toBeOK();
});
test('proxy redirects anonymous users', async ({ page }) => {
  await page.goto('/dashboard');
  await expect(page).toHaveURL('/login?from=%2Fdashboard');
});
```

CODE EXPLAINED

- → production server — realistic caching and rendering
- → form + `toHaveURL` — covers the Server Action end-to-end
- → `request` — Route Handlers directly
- → `proxy.ts` — auth redirects

WHAT CAN BE MOCKED?

REQUEST FROM	MOCK WITH
Browser (client components)	page.route()
Server Components / Actions	Test DB, MSW, testmode
Route Handlers called by browser	page.route() or real
Third-party server APIs	Env var → mock server

NEXT.JS SPECIFICS

TOPIC	NOTE
Caching	Seed before navigation; cached pages may be stale
next/image	Uses /_next/image; works in tests
Hydration	Interact after content is visible
i18n routes	test.use({ locale }) + /[lang] paths

⚠ COMMON MISTAKES

- Mocking `/api/*` with `page.route()` and expecting Server Components to see the mock
- Running tests against `next dev` in CI; on-demand compilation causes timeouts
- Forgetting to rebuild after changing `NEXT_PUBLIC_*` values

✓ TIPS & BEST PRACTICES

- Use a separate test database and reset it in a setup project
- Assert visible results of Server Actions, not network calls
- Use the `with-playwright` example as a starting template

QUICK SUMMARY

- `next build && next start` — test production
- forms — exercise Server Actions
- `request` — Route Handlers
- server fetches — mock outside the browser

Vue, Angular & Svelte Apps

Vue / Nuxt · @nuxt/test-utils/playwright · Angular ng serve :4200 · Protractor migration · SvelteKit vite preview :4173 · hydration waits

OTHER FRAMEWORKS

Playwright tests the rendered page, so the framework only changes **setup**: how the app starts, its port, and how to wait for hydration on SSR frameworks. Vue (via `create-vue`) and SvelteKit (`sv`) scaffolders offer Playwright out of the box; Nuxt adds a helper package with hydration-aware `goto`; Angular uses `ng serve` or a schematic that wires `ng e2e`.

NOTES

- `npm create vue@latest` → choose Playwright for e2e
- `npx sv create` or `npx sv add playwright` for SvelteKit
- `ng add playwright-ng-schematics` → `ng e2e` runs Playwright
- Protractor is end-of-life; Playwright is a common replacement

SETUPS

```
// Vue (create-vue): vite preview on 4173 in CI, dev on 5173 locally
webServer: { command: process.env.CI ? 'npm run preview' : 'npm run dev',
             port: process.env.CI ? 4173 : 5173 },

// Nuxt — hydration-aware helpers
import { expect, test } from '@nuxt/test-utils/playwright';
test.use({ nuxt: { rootDir: fileURLToPath(new URL('..', import.meta.url)) } });
test('home', async ({ page, goto }) => {
  await goto('/', { waitUntil: 'hydration' });
  await expect(page.getByRole('heading')).toHaveText('Welcome to Nuxt');
});

// Angular
webServer: { command: 'npx ng serve', url: 'http://localhost:4200' },

// SvelteKit (sv scaffold)
webServer: { command: 'npm run build && npm run preview', port: 4173 },
```

CODE EXPLAINED

- → Vue — Vite ports differ dev vs preview
- → Nuxt `goto(..., { waitUntil: 'hydration' })` — wait until interactive
- → Angular — `ng serve` on 4200
- → SvelteKit — test the built preview

FRAMEWORK GOTCHAS

FRAMEWORK	WATCH FOR
Vue / Nuxt	Clicks before hydration do nothing
Angular	Material overlays render in a CDK container
Angular	Avoid relying on zone stability; assert UI
SvelteKit	Form actions + progressive enhancement
SvelteKit	Clicks before hydration may do full posts

COMPONENT LIBRARY LOCATORS

LIBRARY	TIP
Angular Material select	getByRole('combobox') → option
Vuetify / Quasar	Prefer labels over generated ids
Svelte transitions	reducedMotion: 'reduce'

⚠ COMMON MISTAKES

- Interacting with SSR pages before hydration; the click lands on static HTML and is lost
- Porting Protractor's `browser.waitForAngular()` habits; Playwright auto-waits on the DOM instead
- Using generated class names (`.v-btn__content` , `.mat-mdc-…`) as locators

✓ TIPS & BEST PRACTICES

- Wait for an interactive signal (enabled button, hydrated marker) before acting
- Use `reducedMotion: 'reduce'` to speed through framework transitions
- Keep each framework's start command in `package.json` scripts and reference them

QUICK SUMMARY

- Vue — create-vue + Vite ports
- Nuxt — `@nuxt/test-utils/playwright`
- Angular — `ng serve` :4200
- SvelteKit — build + preview :4173

Component Testing — Setup

THE STORIES-AND-GALLERY MODEL

Since 1.62, component testing needs **no extra package**. You write small **story** components next to your components, and a **gallery** page (served by your own Vite/dev server) exposes `window.mount({ story, props })`. The built-in `mount` fixture opens the gallery, mounts the story and returns a Locator for its root. Any framework works if your dev server can render the gallery.

NOTES

- `@playwright/experimental-ct-*` packages are discontinued
- `npx playwright init-skills` installs a skill that can generate the gallery
- Gallery exposes `window.mount()` / `window.unmount()` into `#root`

PLAYWRIGHT.CONFIG.TS + GALLERY (REACT, VITE)

```ts
// playwright.config.ts
projects: [{
  name: 'components',
  testDir: './tests/components',
  use: {
    baseURL: 'http://localhost:5173/playwright/gallery/index.html',
    serviceWorkers: 'block', reuseContext: true,
  },
}],
webServer: { command: 'npm run dev', url: 'http://localhost:5173' },

// playwright/gallery/main.tsx  (index.html has <div id="root"> + this script)
import { createRoot } from 'react-dom/client';
const modules = import.meta.glob('/src/**/*.story.tsx', { eager: true });
const root = createRoot(document.getElementById('root')!);
window.mount = async ({ story, props = {} }) => {           // 'components/Button/Primary'
  const i = story.lastIndexOf('/');
  const mod: any = modules[`/src/${story.slice(0, i)}.story.tsx`];
  const Story = mod?.[story.slice(i + 1)];
  if (!Story) throw new Error(`Unknown story: ${story}`);
  root.render(<Story {...props} />);
};
window.unmount = async () => root.render(null);
```

CODE EXPLAINED

- → `components` project — own testDir and baseURL
- → `baseURL` — the gallery page
- → `reuseContext` — fast, shared context
- → `import.meta.glob` — collect stories (Vite)
- → `window.mount` — render a story by id

MIGRATING FROM EXPERIMENTAL-CT

OLD	NEW
`experimental-ct-react/vue`	`@playwright/test` (built-in mount)
`mount(<Button />)`	`mount('components/Button/Primary')`
`JSX in test file`	JSX in *.story.tsx
`playwright/index.tsx`	playwright/gallery/ (yours)

FRAMEWORK SUPPORT

FRAMEWORK	GALLERY RENDERS WITH
React	createRoot().render
Vue	createApp(Story).mount
Svelte	mount(Story, { target })
Solid	render(() => Story)

⚠ COMMON MISTAKES

- Keeping `@playwright/experimental-ct-*` ; packages are no longer published or updated
- Pointing `baseURL` at the app root instead of the gallery page
- Importing app-wide side effects (analytics, service workers) into the gallery

✓ TIPS & BEST PRACTICES

- Stay on 1.62 until you migrate, then move scenario setup into stories
- Load global CSS and providers in the gallery so components render as in the app
- Run the components project separately: `--project=components`

QUICK SUMMARY

- stories — scenarios as components
- gallery — your page exposing `window.mount`
- `mount` fixture — built in, returns Locator
- `components` project — config entry point

Component Testing — Stories & mount

WRITING STORIES AND COMPONENT TESTS

A **story** is a tiny wrapper that sets up one scenario with fixed props, mock data, context providers or recorded callbacks. Each named export is one story; its **ID** is the file path under `src/` without `.story` plus the export name, e.g. `components/Button/Primary`. Tests call `mount(id)` and use normal locators and assertions on the returned root Locator.

NOTES

- Unique suffixes work as IDs: `Button/Primary`
- `mount<typeof Story>(…)` type-checks props against the story
- `component.update(props)` re-renders without remounting
- Unknown story or render error → `mount()` throws

SRC/COMPONENTS/BUTTON.STORY.TSX

```tsx
import { Button } from './Button';
import { ThemeProvider } from '../theme';

export const Primary = () => <Button title="Submit" />;
export const Disabled = () => <Button title="Submit" disabled />;
export const Dark = () => (
  <ThemeProvider mode="dark"><Button title="Submit" /></ThemeProvider>
);

// record callbacks on window so tests can read them
const w = window as any;
export const Clickable = () => (
  <Button title="Go" onClick={() => { w.clicks = (w.clicks ?? 0) + 1; }} />
);
```

STORY PATTERNS

- → one export = one scenario
- → providers wrap the component
- → callbacks stored on `window` for assertions

TESTS/COMPONENTS/BUTTON.SPEC.TS

```ts
import { test, expect } from '@playwright/test';

test('renders primary button', async ({ mount }) => {
  const component = await mount('components/Button/Primary');
  await expect(component.getByRole('button')).toHaveText('Submit');
});

test('click fires onClick', async ({ mount, page }) => {
  const component = await mount('Button/Clickable');            // suffix id
  await component.getByRole('button').click();
  expect(await page.evaluate(() => (window as any).clicks)).toBe(1);
});
```

TEST PATTERNS

- → `mount(id)` — Locator for the root
- → suffix IDs — shorter when unique
- → `page.evaluate` — read recorded callbacks
- → `toHaveScreenshot` works per component

⚠ COMMON MISTAKES

- Passing JSX to `mount()` like the old experimental API; pass a story ID
- Putting test logic in stories; stories set up state, tests act and assert
- Two stories with the same suffix and using the short ID; the lookup is ambiguous

✓ TIPS & BEST PRACTICES

- Co-locate `*.story.tsx` with the component it covers
- Add visual checks: `await expect(component).toHaveScreenshot()`
- Reuse stories in Storybook-like tools for documentation

QUICK SUMMARY

- `*.story.tsx` — named export per scenario
- story ID — path under src + export
- `mount(id)` — root Locator
- `update(props)` — re-render in place

Python — Setup, Sync & Async API

pip install pytest-playwright · playwright install · sync_playwright · async_playwright · snake_case API · expect

PLAYWRIGHT FOR PYTHON

The Python package mirrors the Node API with **snake_case** names and keyword arguments: `get_by_role("button", name="Save")`. It has a **sync API** (simplest, used with pytest) and an **async API** for asyncio programs. Web-first assertions come from `playwright.sync_api.expect`. Install the pytest plugin even for scripts; it pulls in `playwright` itself.

NOTES

- Python 3.9+; `pip install pytest-playwright` (or `uv add`)
- `playwright install --with-deps chromium` downloads browsers
- No `await` in the sync API; methods block
- Timeouts in ms, same as Node

SYNC API (SCRIPT)

```python
import re
from playwright.sync_api import sync_playwright, expect

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False, slow_mo=100)
    page = browser.new_page(viewport={"width": 1280, "height": 720}, locale="en-GB")
    page.goto("https://playwright.dev")
    expect(page).to_have_title(re.compile("Playwright"))
    page.get_by_role("link", name="Get started").click()
    expect(page.get_by_role("heading", name="Installation")).to_be_visible()
    browser.close()
```

CODE EXPLAINED

- → `sync_playwright()` — context manager starts driver
- → `launch(headless=False, slow_mo=…)` — kwargs
- → `new_page(viewport=…, locale=…)` — kwargs options
- → `expect(...).to_have_title()` — snake_case matchers
- → `re.compile` — RegExp equivalent

ASYNC API

```python
import asyncio
from playwright.async_api import async_playwright, expect

async def main():
    async with async_playwright() as p:
        browser = await p.firefox.launch()
        page = await browser.new_page()
        await page.goto("https://example.com")
        await expect(page.get_by_text("Example Domain")).to_be_visible()
asyncio.run(main())
```

CODE EXPLAINED

- → `async_playwright()` — asyncio version
- → `await` every call, including `expect`
- → same names as sync API

NAME MAPPING

NODE	PYTHON
`getByRole`	get_by_role
`toBeVisible()`	to_be_visible()
`waitForResponse`	expect_response (context manager)
`{ timeout: 5000 }`	timeout=5000

⚠ COMMON MISTAKES

- Mixing sync and async APIs in one program; pick one
- Using the sync API inside an asyncio event loop (e.g. Jupyter); use `async_api` there
- Forgetting `playwright install` after `pip install`; browsers aren't bundled

✓ TIPS & BEST PRACTICES

- Use `with page.expect_download() as d:` / `expect_popup()` for event waits
- Run codegen with `--target python-pytest` for ready-made tests
- Use `uv` or venvs to pin the Playwright version per project

QUICK SUMMARY

- `sync_playwright()` — blocking API
- `async_playwright()` — asyncio API
- snake_case + kwargs — Pythonic names
- `expect()` — same web-first assertions

Python — pytest Plugin

PYTEST-PLAYWRIGHT

The **pytest-playwright** plugin provides fixtures (`page`, `context`, `browser`, `new_context` …) and CLI flags for browsers, headed mode, tracing, video and screenshots. Configure context defaults by overriding `browser_context_args` in `conftest.py`, and parallelize with **pytest-xdist**. For async tests use **pytest-playwright-asyncio** with pytest-asyncio.

NOTES

- Test files: `test_*.py` ; functions `test_*`
- CLI flags only affect the built-in `browser` / `context` / `page` fixtures
- `--base-url` comes from `pytest-base-url`
- Async tests: mark with `asyncio(loop_scope="session")`

CONFTEST.PY + TEST

```python
# conftest.py
import pytest

@pytest.fixture(scope="session")
def browser_context_args(browser_context_args):
    return {**browser_context_args, "viewport": {"width": 1440, "height": 900},
            "storage_state": "playwright/.auth/user.json"}

# test_cart.py
from playwright.sync_api import Page, expect

def test_add_to_cart(page: Page):
    page.goto("/products")                                  # uses --base-url
    page.get_by_role("button", name="Add to cart").first.click()
    expect(page.get_by_test_id("cart-count")).to_have_text("1")

@pytest.mark.skip_browser("webkit")
@pytest.mark.browser_context_args(locale="de-DE", timezone_id="Europe/Berlin")
def test_german_dates(page: Page):
    page.goto("/orders")
    expect(page.get_by_text("01.03.2026")).to_be_visible()
```

CODE EXPLAINED

- → `browser_context_args` — session-wide context options
- → `page: Page` — type hint for autocompletion
- → `.first` — property, not method, in Python
- → `skip_browser` / `only_browser` — markers
- → `browser_context_args` marker — per-test override

CLI FLAGS

FLAG	EFFECT
`--browser chromium`	Repeatable: firefox, webkit
`--browser-channel chrome`	Branded browser
`--headed / --slowmo 200`	Visible / slowed
`--tracing retain-on-failure`	Traces
`--base-url URL`	Relative goto()
`-n auto`	Parallel (pytest-xdist)

FIXTURES

FIXTURE	SCOPE
`page / context`	function
`new_context`	function (factory)
`browser / browser_type`	session
`browser_name / is_webkit`	session

⚠ COMMON MISTAKES

- Overriding `browser_context_args` without merging the incoming dict; you lose plugin defaults
- Expecting `--headed` to affect browsers you launch manually
- Very high `-n` values; more workers than cores slows everything down

✓ TIPS & BEST PRACTICES

- Put common flags in `pytest.ini` : `addopts = --browser chromium --tracing retain-on-failure`
- Use `breakpoint()` with `--headed` for interactive debugging
- Run `pytest --numprocesses auto` in CI

QUICK SUMMARY

- `page` fixture — fresh page per test
- `browser_context_args` — context defaults
- markers — per-test browser/context control
- `-n auto` — parallel via xdist

Java — Setup, JUnit & TestNG

PLAYWRIGHT FOR JAVA

The Java binding (`com.microsoft.playwright:playwright`) uses camelCase methods and **options builder objects** instead of object literals. `Playwright.create()` starts the driver; close it with try-with-resources. Assertions come from `PlaywrightAssertions.assertThat`. With **JUnit 5**, `@UsePlaywright` injects `Page`, `BrowserContext` and friends into test methods; TestNG uses manual lifecycle hooks.

NOTES

- Maven artifact: `com.microsoft.playwright` : `playwright` : `1.64.0`
- CLI = main class `com.microsoft.playwright.CLI` (via `mvn exec:java`)
- Playwright objects are not thread-safe: one per thread
- Roles are enums: `AriaRole.BUTTON`

MAVEN + JUNIT 5 TEST

```java
import com.microsoft.playwright.*;
import com.microsoft.playwright.junit.*;
import com.microsoft.playwright.options.AriaRole;
import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(LoginTest.CustomOptions.class)
public class LoginTest {
  public static class CustomOptions implements OptionsFactory {
    @Override public Options getOptions() {
      return new Options().setHeadless(true).setBaseUrl("http://localhost:3000");
    }
  }
  @Test
  void signsIn(Page page) {                                  // injected fixture
    page.navigate("/login");
    page.getByLabel("Email").fill("ada@example.com");
    page.getByRole(AriaRole.BUTTON,
        new Page.GetByRoleOptions().setName("Sign in")).click();
    assertThat(page).hasURL("http://localhost:3000/dashboard");
  }
}
```

CODE EXPLAINED

- → `@UsePlaywright` — JUnit 5 extension
- → `OptionsFactory` — headless, baseUrl, context
- → `Page page` param — injected per test
- → `GetByRoleOptions().setName()` — options builder
- → `assertThat(...).hasURL()` — web-first assertion

NAME MAPPING

NODE	JAVA
`page.goto()`	page.navigate()
`toHaveText()`	hasText()
`toBeVisible()`	isVisible()
`{ name: 'x' }`	new GetByRoleOptions().setName("x")

TESTNG / MANUAL LIFECYCLE

HOOK	CREATE
`@BeforeClass`	Playwright.create(), browser
`@BeforeMethod`	browser.newContext(), newPage()
`@AfterMethod / Class`	close() each

⚠ COMMON MISTAKES

- Sharing one `Playwright` instance across parallel threads; create one per thread
- Forgetting to close `Playwright`; driver processes leak
- Using `page.goto` from TS examples; Java uses `navigate`

✓ TIPS & BEST PRACTICES

- Enable JUnit parallel execution; `@UsePlaywright` gives each thread its own objects
- Use `PLAYWRIGHT_JAVA_SRC` so codegen-generated traces link to sources
- Generate code with `codegen --target java-junit`

QUICK SUMMARY

- `@UsePlaywright` — JUnit 5 fixtures
- options builders — `.setX()` chains
- `assertThat()` — web-first assertions
- TestNG — manual create/close hooks

.NET — Setup & API

PLAYWRIGHT FOR .NET

The .NET binding uses **PascalCase** methods ending in **Async**, options objects initialised with `new() { … }`, and `Expect()` for web-first assertions. Browsers are installed with the generated `playwright.ps1` script after building the project. Use the library package for scripts and tools, or a test-framework package (NUnit, MSTest, xUnit) for tests (next sheet).

NOTES

- .NET 8+; PowerShell (`pwsh`) runs the install script on any OS
- `Microsoft.Playwright` = library; `.NUnit/.MSTest/.Xunit` = test helpers
- Roles via `AriaRole.Button` enum
- `await using` disposes browsers (IAsyncDisposable)

CONSOLE APP

```
// dotnet new console -n Scraper && cd Scraper
// dotnet add package Microsoft.Playwright && dotnet build
// pwsh bin/Debug/net8.0/playwright.ps1 install --with-deps chromium
using System.Text.RegularExpressions;
using Microsoft.Playwright;
using static Microsoft.Playwright.Assertions;

using var playwright = await Playwright.CreateAsync();
await using var browser = await playwright.Chromium.LaunchAsync(
    new() { Headless = false, SlowMo = 50 });
var context = await browser.NewContextAsync(new() { Locale = "en-US" });
var page = await context.NewPageAsync();

await page.GotoAsync("https://playwright.dev");
await Expect(page).ToHaveTitleAsync(new Regex("Playwright"));
await page.GetByRole(AriaRole.Link, new() { Name = "Get started" }).ClickAsync();
await Expect(page.GetByText("Installation").First).ToBeVisibleAsync();
await page.ScreenshotAsync(new() { Path = "docs.png", FullPage = true });
```

CODE EXPLAINED

- → `Playwright.CreateAsync()` — start driver
- → `new() { Headless = false }` — target-typed options
- → `…Async()` — every call is awaited
- → `Expect(...)` — from `Assertions` static import
- → `AriaRole.Link` — role enum

NAME MAPPING

NODE	C#
`page.goto(url)`	await page.GotoAsync(url)
`locator.click()`	await locator.ClickAsync()
`expect(l).toHaveText()`	await Expect(l).ToHaveTextAsync()
`{ name: 'x' }`	new() { Name = "x" }
`page.on('dialog', fn)`	page.Dialog += (_, d) => …

CLI VIA PLAYWRIGHT.PS1

COMMAND	PURPOSE
`install`	Download browsers
`install --with-deps`	+ OS packages
`codegen URL`	Record C# code
`show-trace trace.zip`	Open a trace

⚠ COMMON MISTAKES

- Running `playwright.ps1` before `dotnet build` ; the script doesn't exist yet
- Forgetting `await` on `Expect(...)` ; assertions silently don't run
- Disposing the browser while pages are still in use

✓ TIPS & BEST PRACTICES

- Set `PLAYWRIGHT_BROWSERS_PATH` in CI so browsers are cached between runs
- Use `codegen --target csharp-nunit` for test scaffolding
- Use events (`page.Request +=`) for network logging

QUICK SUMMARY

- `Microsoft.Playwright` — .NET package
- `…Async()` + `await` — every call
- `new() { … }` — options objects
- `Expect()` — web-first assertions

.NET — NUnit, MSTest & xUnit

TEST FRAMEWORK INTEGRATIONS

`Microsoft.Playwright.NUnit`, `.MSTest` and `.Xunit` provide base classes (`PageTest`, `ContextTest`, `BrowserTest`) that create isolated browser contexts per test and expose `Page`, `Context` and `Expect()`. Override `ContextOptions()` for context settings, and configure browser, headless mode and timeouts in a **.runsettings** file or with environment variables.

NOTES

- NUnit: `[Parallelizable(ParallelScope.Self)]` for parallel fixtures
- xUnit: `Microsoft.Playwright.Xunit` (v3: `.Xunit.v3`)
- `BROWSER=webkit dotnet test` switches engine

NUNIT TEST + .RUNSETTINGS

```
using System.Text.RegularExpressions;
using Microsoft.Playwright;
using Microsoft.Playwright.NUnit;

[Parallelizable(ParallelScope.Self)]
public class CheckoutTests : PageTest
{
    public override BrowserNewContextOptions ContextOptions() => new()
    {
        BaseURL = "http://localhost:3000",
    };

    [Test]
    public async Task AddsItemToCart()
    {
        await Page.GotoAsync("/products");
        await Page.GetByRole(AriaRole.Button, new() { Name = "Add to cart" })
                  .First.ClickAsync();
        await Expect(Page.GetByTestId("cart-count")).ToHaveTextAsync("1");
    }
}
// dotnet test --settings test.runsettings   (BrowserName, Headless, ExpectTimeout)
```

CODE EXPLAINED

- → `PageTest` — fresh context + Page per test
- → `ContextOptions()` — BaseURL, locale, color scheme
- → `Page` / `Expect` — inherited members
- → `.First` — property in .NET
- → `.runsettings` — browser, headless, timeouts

PACKAGES & BASE CLASSES

PACKAGE	BASE CLASSES
`Microsoft.Playwright.NUnit`	PageTest, ContextTest, BrowserTest
`Microsoft.Playwright.MSTest`	PageTest, ContextTest, BrowserTest
`Microsoft.Playwright.Xunit`	PageTest (+ .v3 for xUnit v3)

SETTINGS & ENV VARS

SETTING	EFFECT
`BROWSER`	chromium / firefox / webkit
`HEADED=1`	Show browser
`<ExpectTimeout>`	Assertion timeout
`<LaunchOptions>`	Channel, headless, slowMo

⚠ COMMON MISTAKES

- Creating `Playwright` manually inside a `PageTest`; the base class already manages it
- Forgetting `[Parallelizable]`; NUnit runs fixtures sequentially by default
- Hard-coding absolute URLs instead of `BaseURL` in `ContextOptions()`

✓ TIPS & BEST PRACTICES

- Keep separate `.runsettings` files for local (headed) and CI
- Use `Context.Tracing.StartAsync` in `[SetUp]` for traces on failure
- Generate tests with `codegen --target csharp-nunit` or `csharp-mstest`

QUICK SUMMARY

- `PageTest` — per-test Page
- `ContextOptions()` — context config
- `.runsettings` — browser + timeouts
- `BROWSER` env — choose engine

Library Mode (No Test Runner)

PLAYWRIGHT AS A LIBRARY

The `playwright` package exposes the browser API without the test runner: no fixtures, no retries, no reports. Use it for scripts, scraping, PDF generation, screenshots in back-end services, or to embed Playwright in another framework. You manage the lifecycle yourself; since 1.59 many objects are **async disposables**, so `await using` closes them automatically.

NOTES

- `npm i playwright` (downloads browsers) or `playwright-core` (doesn't)
- `expect` web-first assertions come from `@playwright/test`
- `launchPersistentContext()` keeps a real profile on disk
- PDF generation: Chromium only (`page.pdf()`)

SCRIPT

```
import { chromium, devices } from 'playwright';

await using browser = await chromium.launch({ headless: true });      // auto-close
await using context = await browser.newContext({ ...devices['Desktop Chrome'] });
const page = await context.newPage();

await page.goto('https://news.ycombinator.com');
const titles = await page.locator('.titleline > a').allTextContents();
console.log(titles.slice(0, 5));

await page.pdf({ path: 'front-page.pdf', format: 'A4' });             // Chromium only
await page.screenshot({ path: 'front-page.png', fullPage: true });

// persistent profile (extensions, logged-in sessions)
const ctx = await chromium.launchPersistentContext('./profile', { headless: false });
await ctx.close();
```

CODE EXPLAINED

- → `chromium.launch()` — start a browser
- → `await using` — dispose at scope end (1.59)
- → `allTextContents()` — scrape a list
- → `page.pdf()` — print to PDF
- → `launchPersistentContext` — real user-data dir

LIBRARY VS TEST RUNNER

FEATURE	PLAYWRIGHT · @PLAYWRIGHT/TEST
`Fixtures / isolation`	Manual · Automatic
`Retries, reports, traces`	Manual · Built in
`Parallelism`	Your code · Workers
`Web-first expect`	Import · Built in
`Best for`	Scripts, services · Tests

ENTRY POINTS

API	RETURNS
`chromium / firefox / webkit`	BrowserType
`browserType.launch()`	Browser
`launchPersistentContext()`	BrowserContext
`browserType.connect(ws)`	Remote Browser
`request.newContext()`	API client

⚠ COMMON MISTAKES

- Forgetting `browser.close()` without `await using` ; Node never exits
- Rebuilding a test runner on top of the library; use `@playwright/test`
- Scraping sites against their terms or robots rules; check before automating

✓ TIPS & BEST PRACTICES

- Use `playwright-core` in Docker images that already ship browsers
- Reuse one browser and create a context per job in services
- Add `context.tracing` for debugging scripts too

QUICK SUMMARY

- `playwright` — library only
- `launch()` → `newContext()` → `newPage()`
- `await using` — auto cleanup
- `page.pdf()` / screenshots — headless rendering

Jest, Vitest & BDD

PLAYWRIGHT UNDER OTHER RUNNERS

Vitest 4 browser mode runs component and unit tests in a real browser using Playwright as the provider. **jest-playwright** is no longer actively maintained; new Jest users should move e2e tests to `@playwright/test`. For **BDD**, `playwright-bdd` compiles Gherkin `.feature` files into Playwright tests, so you keep fixtures, parallelism and reports.

NOTES

- `npm i -D vitest @vitest/browser-playwright`
- Vitest tests import `page` from `vitest/browser`
- `npx bddgen` generates specs from features before `playwright test`
- Cucumber.js + library mode also works, but loses runner features

VITEST BROWSER MODE + PLAYWRIGHT-BDD

```ts
// vitest.config.ts
import { defineConfig } from 'vitest/config';
import { playwright } from '@vitest/browser-playwright';
export default defineConfig({
  test: { browser: { enabled: true, provider: playwright(),
                     instances: [{ browser: 'chromium' }, { browser: 'webkit' }] } },
});
// playwright.config.ts with playwright-bdd
import { defineBddConfig } from 'playwright-bdd';
const testDir = defineBddConfig({ features: 'features/*.feature',
                                  steps: 'features/steps/*.ts' });
export default defineConfig({ testDir });
// features/steps/cart.ts
import { createBdd } from 'playwright-bdd';
const { Given, When, Then } = createBdd();
Given('I am on the shop', async ({ page }) => { await page.goto('/'); });
When('I add {string} to the cart', async ({ page }, item: string) => {
  await page.getByRole('button', { name: `Add ${item}` }).click();
});
Then('the cart shows {int} item(s)', async ({ page }, n: number) =>
  expect(page.getByTestId('cart-count')).toHaveText(String(n)));
```

CODE EXPLAINED

- → `provider: playwright()` — Vitest drives Playwright
- → `instances` — browsers to run
- → `defineBddConfig` — features + steps → generated tests
- → `createBdd()` — Given/When/Then with fixtures
- → `{string}` / `{int}` — Cucumber expressions

GHERKIN FEATURE

LINE	EXAMPLE
`Feature:`	Shopping cart
`Scenario:`	Add one item
`Given`	I am on the shop
`When`	I add "Mug" to the cart
`Then`	the cart shows 1 item

WHICH RUNNER?

GOAL	USE
`E2E tests`	@playwright/test
`Component/unit in browser`	Vitest browser mode
`Business-readable specs`	playwright-bdd
`Legacy Jest e2e`	Migrate to @playwright/test

⚠ COMMON MISTAKES

- Running `playwright test` without `bddgen` first; generated specs are stale or missing
- Using Vitest browser mode for full e2e flows; it's built for component/unit scope
- Mixing Jest's `expect` with Playwright's in one file

✓ TIPS & BEST PRACTICES

- Add `"test:bdd": "bddgen && playwright test"` to scripts
- Keep step definitions thin; call page objects
- Share locators between Vitest component tests and Playwright e2e via roles

QUICK SUMMARY

- Vitest — `@vitest/browser-playwright` provider
- jest-playwright — migrate away
- playwright-bdd — Gherkin on Playwright
- `bddgen` — generate before running

Cross-Language API Map

TS vs Python vs Java vs C# naming · getByRole / get_by_role / GetByRole · expect forms · config locations

ONE API, FOUR LANGUAGES

All bindings wrap the same engine and share concepts, defaults and behaviour. Only naming and option syntax change: **camelCase** with object literals (TS), **snake_case** with keyword args (Python), **camelCase** with option builders (Java) and **PascalCase**`Async` with `new() { }` initialisers (C#). Use this table to translate examples from the docs.

NOTES

- Only TS has the full `@playwright/test` runner features (projects, UI mode, sharding)
- Python/Java/.NET use pytest, JUnit/TestNG, NUnit/MSTest/xUnit
- Trace viewer, codegen and Inspector work for all
- Versions are released in lockstep

API TRANSLATION

TYPESCRIPT	PYTHON	JAVA	C#
`page.goto(url)`	`page.goto(url)`	`page.navigate(url)`	`page.GotoAsync(url)`
`getByRole('button', { name })`	`get_by_role("button", name=…)`	`getByRole(AriaRole.BUTTON, opts.setName(…))`	`GetByRole(AriaRole.Button, new() { Name = … })`
`locator.fill('x')`	`fill("x")`	`fill("x")`	`FillAsync("x")`
`locator.first()`	`.first`	`.first()`	`.First`
`expect(l).toBeVisible()`	`expect(l).to_be_visible()`	`assertThat(l).isVisible()`	`Expect(l).ToBeVisibleAsync()`
`expect(l).toHaveText('x')`	`to_have_text("x")`	`hasText("x")`	`ToHaveTextAsync("x")`
`expect(l).not.toBeChecked()`	`not_to_be_checked()`	`not().isChecked()`	`Not.ToBeCheckedAsync()`
`waitForEvent('download')`	`with page.expect_download() as d:`	`page.waitForDownload(() -> …)`	`RunAndWaitForDownloadAsync(…)`
`page.on('dialog', fn)`	`page.on("dialog", fn)`	`page.onDialog(d -> …)`	`page.Dialog += (_, d) => …`
`{ timeout: 5000 }`	`timeout=5000`	`.setTimeout(5000)`	`new() { Timeout = 5000 }`

RUNNERS & CONFIG

LANGUAGE	RUNNER · CONFIG LOCATION
TypeScript / JS	@playwright/test · `playwright.config.ts`
Python	pytest-playwright · `pytest.ini`, `conftest.py`, CLI flags
Java	JUnit 5 `@UsePlaywright` · `OptionsFactory`
.NET	NUnit / MSTest / xUnit · `.runsettings`, `ContextOptions()`

CODEGEN --TARGET

LANGUAGE	TARGET
TS test	playwright-test
Python	python-pytest
Java	java-junit
C#	csharp-nunit

⚠ COMMON MISTAKES

- Copying TS option objects into Java/C#; each language has its own options syntax
- Expecting `projects`, `webServer` or UI mode in Python/Java/.NET; they're TS-runner features
- Calling `first()` as a method in Python/C#; it's a property there

✓ TIPS & BEST PRACTICES

- Switch the docs language selector on playwright.dev to see every example in your language
- Use codegen with the right `--target` to learn idioms quickly
- Keep locator strategies identical across language suites

QUICK SUMMARY

- TS — camelCase + objects
- Python — snake_case + kwargs
- Java — builders + `assertThat`
- C# — `Async` + `new() { }`

GitHub Actions

RUNNING IN GITHUB ACTIONS

`npm init playwright` can generate `.github/workflows/playwright.yml`. The essentials: install dependencies, install browsers **with OS deps**, run tests, and **always** upload the report so failures can be inspected. For large suites, a matrix of shards writes **blob reports** that a final job merges into one HTML report.

NOTES

- `if: ${{ !cancelled() }}` uploads reports even when tests fail
- Set `CI=true` (Actions does) so config picks CI settings
- The `github` reporter adds failure annotations to PRs
- Alternatively run jobs in the Playwright container image

.GITHUB/WORKFLOWS/PLAYWRIGHT.YML (SHARDED)

```yaml
name: Playwright Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 60
    strategy: { fail-fast: false, matrix: { shard: [1/4, 2/4, 3/4, 4/4] } }
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v5
        with: { node-version: lts/*, cache: npm }
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test --shard=${{ matrix.shard }}
      - uses: actions/upload-artifact@v4
        if: ${{ !cancelled() }}
        with: { name: "blob-${{ strategy.job-index }}", path: blob-report }
  merge:
    if: ${{ !cancelled() }}
    needs: [test]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v5
      - run: npm ci
      - uses: actions/download-artifact@v4
        with: { path: all-blob-reports, pattern: blob-*, merge-multiple: true }
      - run: npx playwright merge-reports --reporter html ./all-blob-reports
      - uses: actions/upload-artifact@v4
        with: { name: html-report, path: playwright-report, retention-days: 14 }
```

CODE EXPLAINED

- → matrix `shard` — 4 parallel machines
- → `fail-fast: false` — let all shards finish
- → `--with-deps` — Linux packages for browsers
- → blob artifact per shard
- → `merge` job — one combined HTML report
- → `!cancelled()` — run even after failures

⚠ COMMON MISTAKES

- Uploading `playwright-report` only on success; you lose the report when you need it most
- Sharding without `reporter: 'blob'` on CI; there's nothing to merge
- Caching `~/.cache/ms-playwright` with a key that ignores the Playwright version

✓ TIPS & BEST PRACTICES

- Use `reporter: process.env.CI ? 'blob' : 'html'` in config
- Run a quick `--only-changed=origin/main` job on PRs, full suite on main
- Download `html-report` and run `npx playwright show-report` locally

QUICK SUMMARY

- install `--with-deps` — CI browsers
- matrix `--shard` — parallel machines
- blob → `merge-reports` — one report
- upload artifacts with `!cancelled()`

Docker & Other CI

CONTAINERS AND OTHER CI SYSTEMS

The official image `mcr.microsoft.com/playwright:v1.64.0-noble` contains browsers and all OS dependencies, so CI jobs and local runs render identically (important for screenshots). Match the image tag to your `@playwright/test` version. Any CI that can run a container or Node works: GitLab, Jenkins, Azure Pipelines, CircleCI and others follow the same install → test → publish-report pattern.

NOTES

- Use `--ipc=host` (Chromium shared memory) and `--init`
- Image runs as root; for untrusted sites use a non-root user + seccomp
- Image has browsers but **not** your `@playwright/test`; still `npm ci`
- Remote: `run-server` in Docker, connect with `PW_TEST_CONNECT_WS_ENDPOINT`

DOCKER

```
docker run -it --rm --ipc=host --init -v "$PWD":/work -w /work \
  mcr.microsoft.com/playwright:v1.64.0-noble \
  /bin/bash -c "npm ci && npx playwright test"

# browsers in Docker, tests on host (e.g. macOS screenshots that match Linux CI)
docker run -p 3000:3000 --rm --init -it \
  mcr.microsoft.com/playwright:v1.64.0-noble \
  /bin/sh -c "npx -y playwright@1.64.0 run-server --port 3000 --host 0.0.0.0"
PW_TEST_CONNECT_WS_ENDPOINT=ws://127.0.0.1:3000/ npx playwright test
```

CODE EXPLAINED

- → `--ipc=host` — avoids Chromium OOM crashes
- → `-v $PWD` — mount the project
- → `run-server` — remote browser server
- → `PW_TEST_CONNECT_WS_ENDPOINT` — use it from the host

GITLAB CI AND JENKINS

```
# .gitlab-ci.yml
e2e:
  image: mcr.microsoft.com/playwright:v1.64.0-noble
  script: [npm ci, npx playwright test]
  artifacts: { when: always, paths: [playwright-report/], expire_in: 1 week }

# Jenkinsfile (declarative)
# pipeline {
#   agent { docker { image 'mcr.microsoft.com/playwright:v1.64.0-noble' } }
#   stages { stage('e2e') { steps { sh 'npm ci'; sh 'npx playwright test' } } }
# }
```

CI NOTES

- → `when: always` — keep reports on failure
- → Azure Pipelines: NodeTool → `npx playwright install --with-deps` → test → PublishTestResults (JUnit)

⚠ COMMON MISTAKES

- Image tag v1.63 with `@playwright/test` 1.64; browser builds don't match and tests fail to launch
- Omitting `--ipc=host`; Chromium crashes under memory pressure
- Generating screenshot baselines on macOS, comparing in Linux containers

✓ TIPS & BEST PRACTICES

- Pin the image digest or exact tag in CI
- Use the same image locally to update visual baselines
- Publish JUnit to your CI's test tab and the HTML report as an artifact

QUICK SUMMARY

- `mcr.microsoft.com/playwright` — official image
- `--ipc=host --init` — recommended flags
- `run-server` — browsers in Docker, tests anywhere
- same pattern on any CI

Remote & Cloud Browsers

launchServer() · connect() · wsEndpoint · Azure Playwright Workspaces · @azure/playwright · BrowserStack/LambdaTest

RUNNING BROWSERS ELSEWHERE

Tests can drive browsers on another machine: a Playwright **browser server** (`launchServer()` / `run-server`) exposes a WebSocket endpoint that `connect()` or `use.connectOptions` attaches to. Cloud services build on this. **Azure Playwright Workspaces** (Azure App Testing, formerly Microsoft Playwright Testing) runs dozens of cloud browsers in parallel while your tests still run in your CI.

NOTES

- Client and server Playwright versions must match (major.minor)
- `exposeNetwork: '<loopback>'` lets cloud browsers reach your localhost app
- `connectOverCDP` is for existing Chrome, not Playwright servers
- Vendors (BrowserStack, LambdaTest, Sauce) offer SDKs or WS endpoints

SELF-HOSTED SERVER + AZURE

```ts
// server.ts — on the remote machine
import { chromium } from 'playwright';
const server = await chromium.launchServer({ port: 4444, host: '0.0.0.0' });
console.log(server.wsEndpoint());                  // ws://host:4444/<guid>

// playwright.config.ts — connect every test
use: {
  connectOptions: { wsEndpoint: process.env.PW_WS!, exposeNetwork: '<loopback>' },
},

// Azure Playwright Workspaces: npm init @azure/playwright@latest
// playwright.service.config.ts
import { defineConfig } from '@playwright/test';
import { createAzurePlaywrightConfig, ServiceOS } from '@azure/playwright';
import { DefaultAzureCredential } from '@azure/identity';
import config from './playwright.config';
export default defineConfig(config, createAzurePlaywrightConfig(config, {
  os: ServiceOS.LINUX,
  exposeNetwork: '<loopback>',
  credential: new DefaultAzureCredential(),
}));
// PLAYWRIGHT_SERVICE_URL=wss://… \
//   npx playwright test -c playwright.service.config.ts --workers=20
```

CODE EXPLAINED

- → `launchServer()` — expose a browser over WS
- → `connectOptions` — runner connects instead of launching
- → `exposeNetwork` — tunnel to local app
- → `createAzurePlaywrightConfig` — cloud browsers
- → `--workers=20` — parallelism beyond your CPU

REMOTE OPTIONS

OPTION	EFFECT
`wsEndpoint`	Server URL
`headers`	Auth headers for the server
`exposeNetwork`	Hosts reachable from remote browser
`timeout`	Connect timeout

WHERE TO RUN BROWSERS

OPTION	GOOD FOR
`Local workers`	Most teams, small suites
`Docker run-server`	Consistent rendering
`Azure Workspaces`	Large parallel suites
`BrowserStack / LambdaTest`	Real devices, old browsers

⚠ COMMON MISTAKES

- Mismatched client/server versions; the protocol handshake fails
- Forgetting `exposeNetwork`; cloud browsers can't reach `localhost:3000`
- Committing service tokens; use CI secrets or Entra ID credentials

✓ TIPS & BEST PRACTICES

- Start with local workers; move to cloud browsers when CI time hurts
- Keep traces on in cloud runs; they're downloaded to your report
- Use `run-server` in Docker to fix cross-OS screenshot diffs

QUICK SUMMARY

- `launchServer` / `run-server` — browser over WS
- `connectOptions` — runner attaches
- `@azure/playwright` — Azure cloud browsers
- `exposeNetwork` — reach local apps

Quick Reference — Locators, Actions & Assertions

getBy* · filter · click/fill · toBeVisible · toHaveText · toHaveURL · expect.soft

LOCATORS

LOCATOR	FINDS
`getByRole(role, { name })`	By ARIA role + name
`getByLabel('Email')`	Form control by label
`getByPlaceholder('Find')`	Input by placeholder
`getByText('Welcome')`	Element by text
`getByAltText('Logo')`	Image by alt
`getByTestId('cart')`	data-testid
`locator('css')`	Raw CSS / XPath
`frameLocator('iframe')`	Inside iframe

NARROWING

METHOD	EFFECT
`filter({ hasText })`	Contains text
`filter({ has: loc })`	Contains locator
`filter({ hasNot })`	Excludes
`and(loc) / or(loc)`	Intersect / union
`within(container)`	Scope (1.64)
`visible()`	Visible only
`first() / last() / nth(i)`	Position
`loc.getByRole(…)`	Search inside

ACTIONS

ACTION	NOTES
`click() / dblclick()`	Waits, then clicks
`fill(text) / clear()`	Set / empty value
`pressSequentially(t)`	Key by key
`press('Enter')`	Key or chord
`check() / uncheck()`	Checkbox, radio
`selectOption(v)`	Native select
`setInputFiles(p)`	Upload
`dragTo(target)`	Drag and drop

LOCATOR ASSERTIONS (AWAIT)

ASSERTION	PASSES WHEN
`toBeVisible()`	Shown (`toBeHidden` = not)
`toHaveText(t | [..])`	Exact text / list
`toContainText(t)`	Substring
`toHaveValue(v)`	Input value
`toHaveCount(n)`	Number of matches
`toBeEnabled()`	Interactive (`toBeDisabled`)
`toBeChecked()`	Checked
`toHaveAttribute(n, v)`	Attribute

PAGE & API ASSERTIONS

ASSERTION	CHECKS
`toHaveURL(url | /re/)`	Current URL
`toHaveTitle(t)`	Title
`toHaveScreenshot()`	Visual diff
`toMatchAriaSnapshot(y)`	A11y tree
`toBeOK()`	API 2xx
`toEqual / toMatchObject`	Plain values (sync)

ASSERTION MODIFIERS

MODIFIER	EFFECT
`.not`	Negate (still retries)
`expect.soft()`	Continue after failure
`expect.poll(fn)`	Retry a function
`expect(fn).toPass()`	Retry a block
`{ timeout: ms }`	Per-assertion timeout
`expect.configure()`	Preset options

⚠ COMMON MISTAKES

- Plain `expect(await l.textContent())` instead of `await expect(l).toHaveText()`
- Missing `await` before web-first assertions
- `nth()` where a `filter()` by content would be stable

✓ TIPS & BEST PRACTICES

- Priority: role → label → placeholder → text → test id → CSS
- Scope with `filter` / `within` before picking a role
- Raise timeouts per assertion, not globally

QUICK SUMMARY

- `getByRole` — default locator
- `filter / within` — narrow
- actions auto-wait
- `await expect(...)` — auto-retry

Sheet 67

Quick Reference — CLI & Config

test flags · codegen · show-report · trace · defineConfig keys · use options · env vars

COMMANDS

COMMAND	PURPOSE
`npx playwright test`	Run tests
`test --ui`	UI mode
`test --debug`	Inspector
`show-report`	Open HTML report
`show-trace file.zip`	Open a trace
`codegen [url]`	Record a test
`install --with-deps`	Browsers + OS deps
`merge-reports dir`	Combine shards
`init-agents --loop=…`	AI test agents

TEST FLAGS

FLAG	EFFECT
`file.spec.ts:12`	One test by line
`-g / -G "re"`	Grep / invert titles
`--project=name`	Select project
`--headed`	Visible browser
`-j / --workers=N`	Parallelism
`--retries=N`	Retry failures
`--shard=i/n`	Split across machines
`--last-failed`	Rerun failures
`--update-snapshots`	Rewrite baselines

ENVIRONMENT VARIABLES

VARIABLE	EFFECT
`CI`	CI-specific config
`PWDEBUG=1`	Inspector mode
`DEBUG=pw:api`	API logging
`PLAYWRIGHT_BROWSERS_PATH`	Browser cache
`PW_TEST_CONNECT_WS_ENDPOINT`	Remote browsers
`PLAYWRIGHT_HTML_OPEN`	never / always

CONFIG KEYS

KEY	TYPICAL
`testDir`	`'./tests'`
`fullyParallel`	true
`forbidOnly`	`!!process.env.CI`
`retries`	`CI ? 2 : 0`
`workers`	`CI ? '50%' : undefined`
`reporter`	`'html'` / `['blob']`
`timeout`	30_000
`expect.timeout`	5_000
`webServer`	{ command, url }

USE OPTIONS

OPTION	EXAMPLE
`baseURL`	`'http://localhost:3000'`
`trace`	`'on-first-retry'`
`screenshot`	`'only-on-failure'`
`video`	`'retain-on-failure'`
`storageState`	`'.auth/user.json'`
`actionTimeout`	10_000
`locale / timezoneId`	`'en-US'` / `'UTC'`
`...devices[name]`	Device profile

PROJECT KEYS

KEY	PURPOSE
`name`	`--project` target
`use`	Per-project options
`testMatch`	Files for project
`dependencies`	Run after these
`teardown`	Run when done
`default`	false = opt-in (1.64)

⚠ COMMON MISTAKES

- Positional args as titles; they filter **file paths**
- `baseURL` outside `use`
- CI without `--with-deps` on Linux

✓ TIPS & BEST PRACTICES

- Branch config on `process.env.CI`
- `trace: 'on-first-retry'` + HTML report on CI
- Use `--last-failed` for fast iteration

QUICK SUMMARY

- `test`, `show-report`, `codegen`
- `-g`, `--project`, `:line` filters
- `defineConfig` + `use` + `projects`
- `CI`, `PWDEBUG`, `DEBUG` env vars